HERMES AI LAB

Hermes AI 연구실 설치 매뉴얼

Discord · Hermes Agent · 로컬 모델 · Claude Code

처음 해 보는 사람도 그대로 따라 하면 끝나도록 한 단계씩



대상	Linux 서버 한 대에 Discord로 대화하는 AI 연구실(직원 6명)을 새로 만들려는 사람
전제	로컬 모델(Qwen3.6-35B-A3B, llama-server)은 이미 돌고 있다 (`LOCAL_MODEL.md`). 작업 서버(예: gnode1)의 홈 `/ai/works/<계정>`에 설치한다
기준	2026년 11월, Hermes Agent v0.22.5. 메뉴 이름과 옵션은 버전에 따라 조금씩 다를 수 있습니다
설정 묶음	`~/Projects/hermes-team/` (이 문서와 함께 배포)

차례

1 시작하기 전에 **1**
- 1.1 이 매뉴얼에서 할 일 . . . 1
- 1.2 큰 그림: 무엇이 어디서 돌아가는가 . . . 2
- 1.3 준비물 . . . 3
- 1.4 이 문서를 읽는 법 . . . 4
- 1.5 터미널 기초 (처음이라면) . . . 4
- 1.6 ID 기록표 . . . 5
- 1.7 설치 위치 확인: 홈이 /ai/works에 있는지 . . . 6
  - 1.7.1 gnode1에 접속하고 홈 위치 확인 . . . 6
  - 1.7.2 로그인 설정 파일 챙기기 . . . 7
  - 1.7.3 서비스용 홈 확인 . . . 7

2 기본 도구 설치 **9**
- 2.1 기본 패키지와 git . . . 9
- 2.2 uv (프로젝트별 Python 환경) . . . 9
- 2.3 Hermes Agent . . . 10
- 2.4 Claude Code . . . 11
- 2.5 Antigravity CLI . . . 12
- 2.6 연구실 설정 묶음 가져오기 . . . 12
  - 2.6.1 연구자 이름 정하기 . . . 13

3 Discord 서버 준비 **14**
- 3.1 개발자 모드 켜기 . . . 14
- 3.2 서버와 채널 만들기 . . . 14
- 3.3 ID 네 개 복사하기 . . . 14

4 직원 6명 만들기 **15**
- 4.1 Discord 봇 6개 만들기 . . . 15
- 4.2 PI 설정 . . . 16
  - 4.2.1 로컬 모델 서버 확인 . . . 16
  - 4.2.2 주 모델을 로컬 모델로 . . . 17
  - 4.2.3 예비 모델 넣기 . . . 17
  - 4.2.4 Discord 연결 . . . 19
  - 4.2.5 시험 . . . 19
- 4.3 나머지 직원 5명 만들기 . . . 20
  - 4.3.1 profile 만들기 . . . 20
  - 4.3.2 직원마다 자기 봇 토큰 넣기 . . . 21
  - 4.3.3 analyst·reviewer는 ChatGPT로 . . . 21
  - 4.3.4 확인 . . . 23

5 다른 모델 고르기 **24**
- 5.1 바꿀 때마다 지키는 순서 . . . 24

5.2 주 모델 바꾸기 . . . 24
5.3 예비 모델 바꾸기 . . . 25
5.4 그 밖에 바꿀 수 있는 것 . . . 26

6 팀으로 묶기 27
6.1 팀 설정 파일 채우기 . . . 27
6.1.1 team.json . . . 27
6.1.2 souls/_team_ids.md . . . 27
6.2 게이트웨이를 서비스로 켜기 . . . 28
6.3 한 번에 적용하기 . . . 28
6.4 직원 봇의 슬래시 명령 등록 (처음 한 번) . . . 29
6.5 PI에게 채널 만들 권한 주기 . . . 30
6.6 Antigravity 읽기 권한 . . . 30
6.7 자동 장치 설치 (아침 브리핑, 차단 감시, PI 인계) . . . 30
6.8 PI 홈 채널 정하기 . . . 32
6.9 SSH를 끊어도 계속 도는지 확인 . . . 32
6.9.1 서비스가 등록되지 않을 때 (홈 경로 문제) . . . 33

7 점검과 첫 대화 36
7.1 자동 점검 . . . 36
7.2 Discord에서 시험하기 . . . 36
7.3 반응이 없을 때 . . . 37
7.4 예시: 분자 용해도 예측으로 연구 한 바퀴 . . . 38

8 매일 쓰는 법 43
8.1 기본 흐름 . . . 43
8.2 상황별 방법 . . . 43
8.3 프로젝트 폴더 안내 . . . 44
8.3.1 왜 이렇게 나눴나: 연구자의 생각 순서를 폴더로 . . . 44
8.3.2 폴더별 안내 . . . 44
8.4 계산 노드에서 실험 돌리기 (hpc.sh) . . . 46
8.4.1 처음 한 번 . . . 46
8.4.2 시험해 보기 . . . 47
8.4.3 실행 . . . 47
8.5 새 직원 추가하기 . . . 48

9 문제 해결 50
9.1 먼저 해 볼 것 . . . 50
9.2 증상별 해결 . . . 50

A 지금 연구실을 다른 컴퓨터로 옮기기 52

B 다른 사람에게 넘기기 (넘겨주는 쪽) 53

C 명령어와 파일 요약 54
C.1 자주 쓰는 명령 . . . 54
C.2 파일 지도 . . . 55

CHAPTER 1

시작하기 전에

1.1 이 매뉴얼에서 할 일

이 매뉴얼을 끝까지 따라 하면 작업 서버(예: gnode1)에 **AI 직원 6명이 일하는 연구실**이 생깁니다. 직원들은 Discord 서버에 봇으로 들어와 있고, 나는 휴대폰이나 PC의 Discord 앱에서 사람에게 말하듯 일을 맡깁니다. "새 프로젝트 만들자"라고 하면 PI(책임자)가 채널과 폴더를 만들고, 일을 kanban 카드로 나눠 다른 직원에게 맡기고, 결과를 검토해 정리합니다.

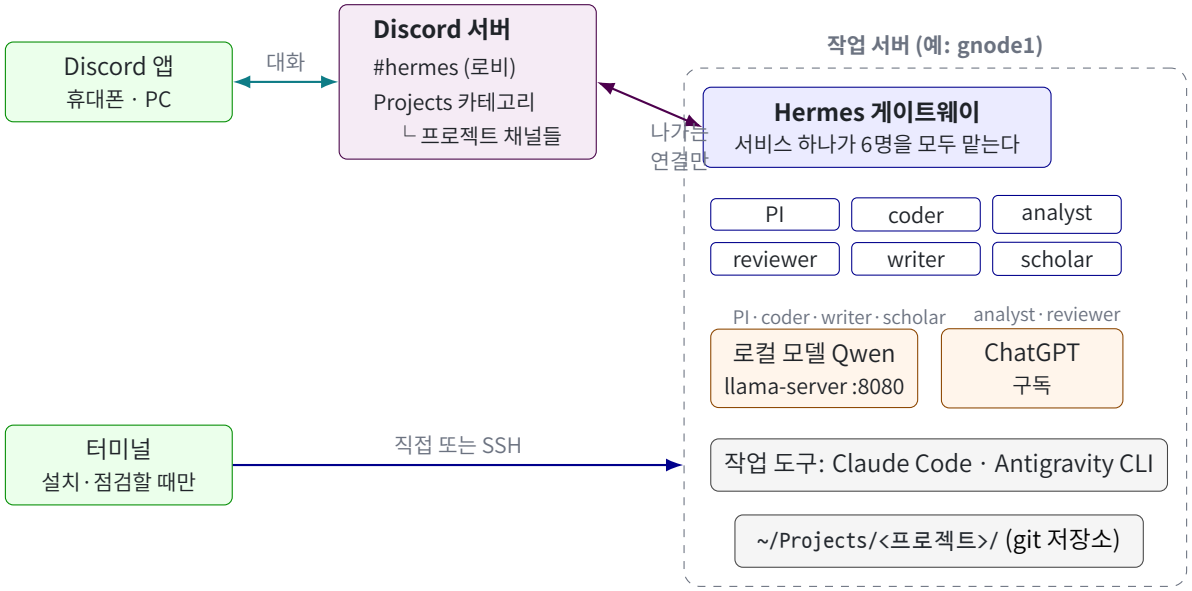
직원	하는 일	두뇌(모델)와 도구
PI	연구 책임자. 일을 나누고 승인·보고	로컬 모델 (Qwen)
coder	코드 작성·수정	로컬 모델 + Claude Code
analyst	실험 실행과 결과 분석	ChatGPT
reviewer	코드·분석·초안 독립 검토	ChatGPT
writer	논문·보고서 초안	로컬 모델 + Claude Code
scholar	문헌 조사와 논문 읽기	로컬 모델 + Antigravity

주 모델이 꺼지거나 한도에 걸리면 6명 모두 **예비 모델 Claude Opus**로 넘어갑니다. 어떤 모델을 쓸지는 직원별로 바꿀 수 있습니다 (5장).

이 매뉴얼을 마치면 할 수 있는 것

- Hermes Agent, Claude Code, Antigravity CLI를 설치하고 각 계정으로 로그인하기
- Discord 봇 6개를 만들어 내 서버에 초대하기
- 직원 6명을 만들고, 그중 4명이 **이미 설치된 로컬 모델**로 생각하게 연결하기
- 직원들을 한 팀으로 묶고, 재부팅해도 계속 돌게 하기
- 자동 점검(`lab.py check`)을 통과하고 Discord에서 첫 프로젝트 만들어 보기

1.2 큰 그림: 무엇이 어디서 돌아가는가



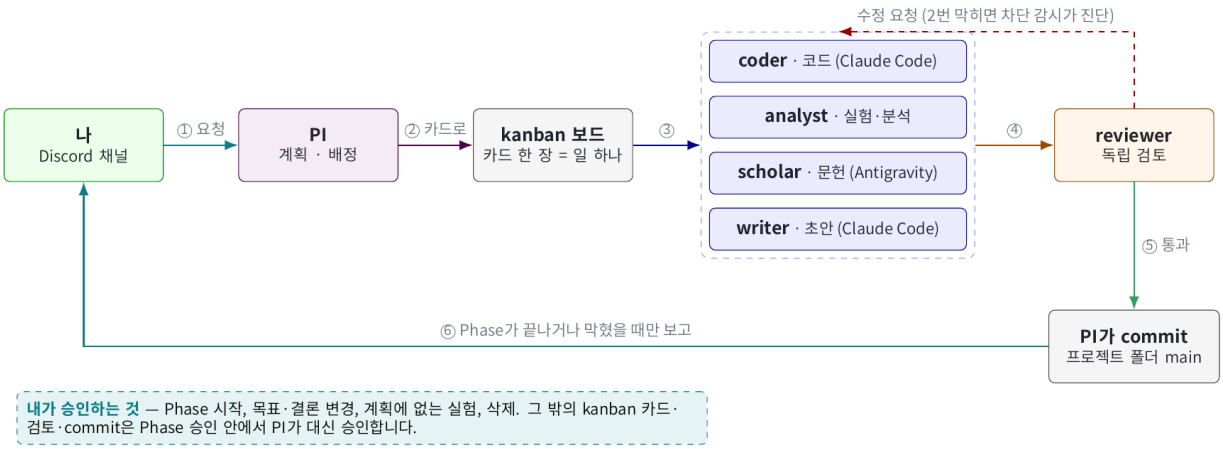
그림에서 기억할 점은 세 가지입니다.

1. **모든 것은 작업 서버 한 대(예: gnode1)에서 돕니다.** Discord는 대화 창구일 뿐입니다. 파일은 그 서버의 로컬 디스크(`/ai/works/<계정>`)에 둡니다 (1.7절). 게이트웨이가 Discord에 나가는 연결을 열어 두므로 공유기 포트를 열 필요가 없습니다.
2. **직원 = Discord 봇 1개 + Hermes profile 1개.** 그래서 봇을 6개 만들고, profile도 6개 만듭니다. PI는 기본 profile(`default`)을 씁니다.
3. **설정의 원본은 `~/Projects/hermes-team/` 폴더입니다.** 직원들의 역할 · 규칙(SOUL), 새 프로젝트 뼈대, 설치 도우미 스크립트가 들어 있습니다. Hermes 자체의 데이터(토큰, 대화 기록)는 `~/.hermes/`에 따로 쌓입니다.

일이 굴러가는 방식

> **kanban이란** 일을 **카드 한 장 = 일 하나**로 적어 보드에 붙여 두고, 카드가 "할 일 → 진행 중 → 완료"로 옮겨 가는 것을 보며 관리하는 방식입니다. Hermes에는 이 kanban 보드가 들어 있고, 프로젝트마다 보드가 하나씩 생깁니다(보드 이름 = 프로젝트 폴더 이름). PI가 kanban 카드를 만들어 직원에게 배정하면, 직원은 자기 카드를 가져가 일하고 결과를 카드에 남깁니다. 터미널에서는 `hermes kanban --board <프로젝트> list`로 볼 수 있습니다.

설치가 끝난 뒤 요청 하나가 처리되는 길은 아래와 같습니다. 나는 PI에게만 말하면 되고, 나머지는 직원들이 kanban 카드로 주고받습니다.



1. 내가 프로젝트 채널에서 PI에게 요청합니다.
2. PI가 일을 **kanban 카드**로 나눠 kanban 보드에 올리고 담당 직원을 정합니다.
3. 각 직원이 자기 kanban 카드를 실행합니다. coder·writer는 Claude Code로, scholar는 Antigravity로 실제 작업을 합니다.
4. 결과는 reviewer가 독립적으로 검토합니다. 문제가 있으면 수정 요청이 돌아가고, 같은 kanban 카드가 2번 막히면 자동 장치(차단 감시)가 원인을 진단합니다.
5. 통과하면 PI가 프로젝트 폴더의 `main`에 commit합니다.
6. PI는 Phase가 끝나거나 막혔을 때만 나에게 보고합니다.

1.3 준비물

항목	어디에 쓰나	비고
Linux 서버 (예: `gnode1`)	연구실 전체가 도는 곳	SSH로 접속해 로그인 서버처럼 씁니다. 인터넷, 내 계정, 로컬 디스크의 홈 `/ai/works/<계정>`
로컬 모델 서버	PI·coder·writer·scholar의 두뇌	**이미 설치되어 있다고 가정합니다.** 주소(포트)만 알면 됩니다
ChatGPT 구독 (Plus/Pro)	analyst·reviewer의 두뇌, 대화 압축 요약	Hermes에서 **OpenAI → ChatGPT/Codex subscription**으로 로그인
Claude 구독 (Pro/Max)	coder·writer의 작성 도구 Claude Code, 6명의 예비 모델	주 모델이 꺼지거나 한도에 걸리면 Claude Opus로 넘어갑니다
Google 계정	scholar의 논문 읽기 도구 Antigravity	
Discord 계정 + 내 서버	직원들과 대화하는 곳	서버 관리자여야 합니다. 없으면 3장에서 만듭니다
메모장	ID와 토큰 기록	1.6절의 기록표를 채웁니다

걸리는 시간 처음이면 2~3시간입니다. 대부분은 Discord 봇 6개를 만드는 시간(4장)입니다. 중간에 쉬어도 됩니다. 각 절 끝의 "성공하면" 상자를 확인하고 넘어가면 어디서 멈췄는지 알 수 있습니다.

검증된 버전

이 문서는 아래 버전에서 끝까지 확인했습니다. 더 새 버전이어도 대개 동작합니다. 명령이 바뀐 것 같으면 <도구> --help로 확인합니다.

도구	버전
Hermes Agent	v0.22.5 (2026.9.24, stable)
Claude Code	2.1.311
Antigravity CLI (agy)	1.4.7
uv	0.13.5
git	2.55.0

1.4 이 문서를 읽는 법

본문에는 다음 상자들이 나옵니다.

```
# 남색 상자: 터미널에 그대로 입력하는 명령. # 뒤는 설명이라 입력하지 않아도 된다
hermes --version
```

파일 이름

흰 상자: 파일에 적는 내용입니다. 제목 줄이 파일 이름입니다.

```
회색 상자: 화면에 나오는 결과 예시입니다. 숫자나 날짜는 조금 달라도 됩니다.
```

Discord에서 보낼 메시지
보라색 상자: Discord 채널에 보내는 메시지입니다.

팁 알아 두면 편한 요령입니다.

주의 실수하기 쉬운 부분이나 비밀 정보 관련 주의 사항입니다.

✓ 성공하면 여기 적힌 결과가 나오면 그 절은 끝입니다. 다음으로 넘어갑니다.

꺾쇠 <...>는 내 값으로 바꿔 넣는 자리입니다. 예를 들어 <내이름>은 `minji`처럼 바꿔 입력합니다. 꺾쇠 기호 자체는 지웁니다.

1.5 터미널 기초 (처음이라면)

이 매뉴얼의 명령은 모두 **터미널**에 입력합니다. Ubuntu에서는 `Ctrl`+`Alt`+`T`로 열립니다. 다른 컴퓨터에서 SSH로 접속해 작업해도 똑같습니다.

하고 싶은 것	방법
명령 붙여 넣기	`Ctrl`+`Shift`+`V` (터미널에서는 `Ctrl`+`V`가 아닙니다)
명령 실행	`Enter`. 여러 줄을 한꺼번에 붙여 넣으면 차례로 실행됩니다
실행 중인 것 멈추기	`Ctrl`+`C`
이전 명령 다시 부르기	`↑`
~의 뜻	내 홈 폴더 (gnode1에서는 /ai/works/<사용자>)
sudo 비밀번호	내 로그인 비밀번호. 입력해도 화면에 **아무것도 표시되지 않는 것이 정상**입니다

nano로 파일 고치기

설정 파일은 nano라는 간단한 편집기로 고칩니다. nano <파일>로 열고, 화살표 키로 움직여 고칩니다.

키	동작
`Ctrl`+`O` 다음 `Enter`	저장
`Ctrl`+`X`	나가기
`Ctrl`+`K`	지금 줄 잘라내기 (여러 번 누르면 여러 줄)
`Ctrl`+`W`	찾기
`Ctrl`+`Shift`+`V`	붙여 넣기

1.6 ID 기록표

설치하는 동안 Discord ID 10여 개와 봇 토큰 6개가 나옵니다. 아래 표를 메모장에 복사해 두고 채워 나갑니다.

메모장 (내 컴퓨터에만 보관)

```
[Discord]
서버(guild) ID        :
로비 채널(#hermes) ID  :
Projects 카테고리 ID   :
내 사용자 ID          :

[봇]                   Application ID      봇(사용자) ID       토큰(비밀)
PI (hermes-researcher) :
coder                  :
analyst                :
reviewer               :
writer                 :
scholar                :
```

토큰은 비밀번호입니다 봇 토큰을 가진 사람은 그 봇으로 무엇이든 할 수 있습니다. 채팅·메일·git·스크린샷에 올리지 마세요. 실수로 노출됐다면 Discord 개발자 페이지에서 **Reset Token**으로 새로 만듭니다.

두 가지 경우 이 문서는 **새 연구실을 처음 만드는 경우**입니다. 이미 돌고 있는 연구실을 다른 컴퓨터로 옮기는 경우는 부록 A만 따라 합니다. 같은 봇 토큰을 두 컴퓨터에서 동시에 쓰면 나중에 켠 쪽이 연결에 실패합니다.

1.7 설치 위치 확인: 홈이 /ai/works에 있는지

이 단계에서 하는 일 작업 서버(gnode1)에서 내 홈 폴더가 그 서버의 로컬 디스크에 잡혀 있는지 확인합니다. **도구를 설치하기 전에** 해야 합니다.

이 연구실은 클러스터의 계산 노드 gnode1을 **두 번째 로그인 서버처럼** 씁니다. 로그인 서버를 거치지 않고 내 PC에서 gnode1(172.27.120.13)로 바로 SSH 접속해 설치하고, Hermes도 gnode1에서만 돌립니다. 클러스터에는 성격이 다른 디스크가 여럿 있으니 먼저 지도를 봅니다.

경로	어디에 있나	이 연구실에서의 쓰임
/ai/works/<계정>	gnode1 로컬 디스크	**gnode1에서의 내 홈(~).** Hermes(~/.hermes), 프로젝트(~/Projects) 모두 여기
/data/datasets	gnode1 로컬 디스크	연구실 공용 데이터셋
/data/<계정>	gnode1 로컬 디스크	계산 노드에서 돌린 실험 결과가 도착하는 곳 (8.4절)
/home/<계정>	로그인 서버의 공유 디스크 (NFS)	다른 노드에서의 홈. gnode1에서는 계산 노드로 파일을 보낼 때 **잠깐 거쳐 가는 곳**으로만 씁니다

공유 디스크(NFS)에 Hermes를 두지 않는 이유는 두 가지입니다.

- Hermes는 대화 기록과 kanban 보드를 데이터베이스 파일로 씁니다. 공유 디스크에서는 이 파일이 잠기거나(database is locked) 느려질 수 있습니다.
- 공유 디스크는 모든 노드가 같이 쓰므로, 프로젝트의 Python 환경(.venv)처럼 큰 파일을 두면 다른 사람까지 느려집니다.

gnode1에서는 관리자가 로그인할 때 홈을 /ai/works/<계정>으로 바꿔 주도록 설정해 두었습니다. 그래서 이 문서의 ~/.hermes, ~/Projects는 그대로 /ai/works/<계정>/ 아래가 됩니다. 링크를 만들거나 스크립트를 고칠 필요가 없습니다.

1.7.1 gnode1에 접속하고 홈 위치 확인

gnode1에는 로그인 서버를 거치지 않고 **내 PC에서 바로** 접속합니다. 주소는 172.27.120.13이고, 포트 번호는 관리자에게 물어봅니다. 내 PC의 터미널(Windows는 PowerShell)에서 <포트>와 <계정>을 바꿔 실행합니다.

```
ssh -p <포트> <계정>@172.27.120.13
```

처음 접속하면 Are you sure you want to continue connecting이 나옵니다. yes를 입력하고 비밀번호를 넣습니다. **이후 이 문서의 모든 명령은 gnode1에서** 실행합니다. 접속했으면 홈 위치를 확인합니다.

```
echo $HOME
df -hT $HOME
```

```
/ai/works/minji
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/sdb1      ext4   15T  2.1T   13T  15% /ai
```

✓ **성공하면** `echo $HOME`이 `/ai/works/<계정>`이고, `df`의 Type이 `ext4`나 `xfs`(로컬 디스크)입니다.

`/home/<계정>`이 나오면 gnode1의 홈 설정이 아직 안 된 계정입니다. 관리자에게 `/ai/works/<계정>` 폴더와 홈 설정을 부탁합니다.

매번 주소와 포트를 치기 귀찮다면 **내 PC**의 `~/.ssh/config` 파일에 아래 네 줄을 넣으면 그다음부터는 `ssh gnode1`만 치면 됩니다.

```
Host gnode1
HostName 172.27.120.13
Port <포트>
User <계정>
```

1.7.2 로그인 설정 파일 챙기기

새 홈은 비어 있어서, 로그인할 때 읽는 설정 파일이 없을 수 있습니다. 특히 `.profile`이 없으면 `.bashrc`가 자동으로 읽히지 않아 매번 `source ~/.bashrc`를 해야 합니다.

```
ls -a ~
```

`.profile`이나 `.bashrc`가 없으면 기본 파일을 복사합니다. `-n`은 "이미 있으면 덮어쓰지 않는다"는 뜻입니다.

```
cp -n /etc/skel/.profile /etc/skel/.bashrc ~/
```

로그인 서버에서 쓰던 `.bashrc`(별칭, conda 설정 등)를 그대로 쓰고 싶다면 대신 그것을 복사합니다.

```
cp /home/$USER/.bashrc ~/.bashrc
```

SSH를 끊었다가 다시 접속합니다.

```
exit
ssh -p <포트> <계정>@172.27.120.13
grep -c bashrc ~/.profile
```

✓ **성공하면** 숫자가 1 이상이면 `.profile`이 `.bashrc`를 읽습니다. 이제 `source ~/.bashrc` 없이도 설정이 적용됩니다.

1.7.3 서비스용 홈 확인

Hermes 게이트웨이는 **systemd 사용자 서비스**로 등록해, SSH를 끊거나 서버가 재부팅돼도 계속 돌게 합니다(6.2절). 그러려면 gnode1이 내 홈을 `/ai/works/<계정>`으로 알고 있어야 합니다. 관리자가 미리 설정해 두었는지 세 가지를 확인합니다.

```
getent passwd $USER
systemctl --user show -p UnitPath | tr ' ' '\n' | grep -c works
loginctl show-user $USER -p Linger
```

```
minji:x:1023:1023:,,,:/ai/works/minji:/bin/bash
3
Linger=yes
```

✓ **성공하면** 첫 줄 끝이 `/ai/works/<계정>:/bin/bash`, 둘째 줄이 `1` 이상, 셋째 줄이 `Linger=yes`입니다. 이 문서는 이 상태를 기준으로 씁니다.

하나라도 다르면 설치는 그대로 진행해도 됩니다(2~5장은 로그인 셸에서만 합니다). 다만 게이트웨이를 서비스로 등록하기 전에 6.9.1절 순서대로 관리자에게 부탁해 고칩니다. 첫 줄은 맞는데 둘째 줄만 `0`이면 관리자 없이 해결됩니다: `systemctl --user exit` 후 SSH를 모두 끊고 다시 접속합니다.

Hermes는 gnode1에서만 켭니다 다른 노드에서는 홈이 `/home/<계정>`이라 `~/.hermes`가 없습니다. 다른 서버에서 `hermes`를 설치하거나 실행하지 마세요. 같은 봇 토큰으로 게이트웨이가 두 개 뜨면 나중에 켠 쪽이 연결에 실패합니다.

로컬 디스크는 백업되지 않는 경우가 많습니다 디스크가 고장 나면 사라집니다. 연구 프로젝트는 git 원격 저장소에 push해 두고, 가끔 `hermes backup`(부록 A)과 `bash tools/memories.sh export`로 설정과 기억을 받아 둡니다.

CHAPTER 2

기본 도구 설치

이 장에서는 터미널에서 도구 다섯 개를 설치하고 각자 로그인합니다. 모두 **일반 사용자**로 실행합니다. `sudo`는 쓰라고 한 곳에서만 씁니다.

2.1 기본 패키지와 git

이 단계에서 하는 일 설치 스크립트를 받는 `curl`, 버전 관리 `git`, 스크립트를 돌리는 `python3`를 준비합니다.

```
git --version && curl --version | head -1 && python3 --version
```

세 줄 모두 버전이 나오면 이미 설치된 것이므로 넘어갑니다. 하나라도 `command not found`이면 설치합니다.

```
sudo apt update && sudo apt install -y git curl python3
```

> **공용 서버에서 sudo 권한이 없다면** 관리자에게 설치를 부탁합니다. 이 문서에서 `sudo`가 필요한 곳은 여기와 1.7.3절(gnode1 홈 설정, linger) 두 곳뿐입니다.

이어서 git에 내 이름을 알려 줍니다. 직원들이 만드는 commit에 이 이름이 남습니다.

```
git config --global user.name  "<이름>"
git config --global user.email "<이메일>"
git config --global init.defaultBranch main
```

마지막 줄은 새 저장소의 기본 브랜치를 `master`가 아니라 `main`으로 만듭니다. 직원들은 `main`을 기준으로 일하므로 꼭 넣습니다.

> ✓ **성공하면** `git config --global --list`에 이름·이메일·`init.defaultbranch=main`이 보입니다.

2.2 uv (프로젝트별 Python 환경)

이 단계에서 하는 일 프로젝트마다 Python 패키지를 따로 관리하는 도구를 설치합니다. 새 프로젝트를 만들 때 직원들이 씁니다.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
source ~/.bashrc
uv --version
```

✓ **성공하면** `uv 0.13.5`처럼 버전이 나옵니다.

conda를 쓰던 사람을 위한 uv

uv는 **프로젝트 폴더마다 conda 환경이 하나씩 붙어 있는 것**이라고 보면 됩니다. 이 연구실의 프로젝트는 conda 대신 uv를 씁니다.

하고 싶은 일	conda	uv (프로젝트 폴더에서)
환경 만들기	`conda create -n myenv`	`uv init` (환경은 폴더 안 `.venv/`)
패키지 설치·제거	`conda install` / `remove`	`uv add` / `uv remove`
환경에서 실행	`conda activate` 후 `python`	`uv run python a.py`
켜 두고 계속 쓰기	`conda activate myenv`	`source .venv/bin/activate`
패키지 목록 저장	`environment.yml` (직접 export)	`pyproject.toml`, `uv.lock` (자동)
환경 지우기	`conda env remove -n myenv`	`rm -rf .venv` (`uv sync`로 다시 만듦)

한 프로젝트에서 패키지를 바꿔도 다른 프로젝트는 영향을 받지 않습니다. 한 번 받은 패키지는 캐시에 남아 다시 설치할 때 몇 초면 됩니다. `uv.lock`에 정확한 버전이 적혀 있어서, 계산 노드에도 같은 환경을 옮길 수 있습니다(8.4절).

conda가 저절로 켜지지 않게 프롬프트 앞에 `(base)`가 보이면 conda가 켜진 상태라 uv 환경과 섞일 수 있습니다. `conda config --set auto_activate_base false`로 꺼 둡니다. conda는 그대로 남고, 필요할 때 `conda activate`로 직접 켭니다. 직원들은 conda를 건드리지 않습니다.

2.3 Hermes Agent

이 단계에서 하는 일 직원들의 몸체인 Hermes Agent를 설치합니다.

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash -s -- --non-interactive
source ~/.bashrc
hermes --version
```

`--non-interactive`는 설치가 끝난 뒤 자동으로 뜨는 설정 마법사와 게이트웨이 서비스 등록을 건너뛰라는 뜻입니다. 모델 설정은 4.2절, 서비스 등록은 6.2절에서 따로 하므로 여기서는 프로그램만 설치합니다. 설치되면 세 곳에 파일이 생깁니다.

- `~/.hermes/hermes-agent/`: 프로그램 코드
- `~/.local/bin/hermes`: 명령
- `~/.hermes/`: 설정, 토큰, 대화 기록 (**비밀 포함**)

How would you like to set up Hermes? 창이 떴다면 `--non-interactive` 없이 설치한 경우입니다. 아무것도 고르지 말고 `Ctrl` + `C`를 눌러 명령 프롬프트로 돌아옵니다(한 번에 안 나가면 몇 번 더). 그다음 위의 `curl` 줄을 그대로 다시 실행합니다. 이미 받은 파일은 건너뛰고 빠르게 끝납니다. 목록 첫 줄의 **Quick Setup (Nous Portal)**은 별도 유료 서비스 로그인이라 이 연구실에서는 쓰지 않습니다.

hermes: command not found가 나오면 `source ~/.bashrc`를 한 번 더 하거나, 터미널을 닫고 새로 엽니다. 그래도 안 되면 `echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc` 후 다시 `source ~/.bashrc`.

✓ **성공하면** `hermes --version`이 `v0.22.5` 이상을 출력합니다.

2.4 Claude Code

이 단계에서 하는 일 coder·writer가 실제 코드와 글을 쓸 때 부르는 도구를 설치하고 Claude 구독 계정으로 로그인합니다.

```
curl -fsSL https://claude.ai/install.sh | bash
source ~/.bashrc
claude
```

처음 실행하면 다음 순서로 진행됩니다.

1. 화면 테마를 고릅니다 (아무거나).
2. 로그인 방법에서 **Claude 구독 계정(Pro/Max)**을 고릅니다.
3. 브라우저가 열리면 로그인하고 승인합니다. 브라우저가 없는 컴퓨터(SSH 접속)라면 화면에 나온 주소를 복사해 내 컴퓨터의 브라우저에 붙여 넣고, 승인 후 나오는 코드를 터미널의 `Paste code here` 자리에 붙여 넣습니다.
4. "이 폴더를 신뢰하겠냐"고 물으면, 지금은 홈 폴더(~)에서 열었으므로 **No**를 고릅니다.
5. `/exit`를 입력해 나옵니다.

이제 직원들이 쓰는 방식(`claude -p`)으로 한 번 시험합니다.

```
claude -p "OK라고만 답해" --max-turns 1
```

```
OK
```

ANTHROPIC_API_KEY가 설정되어 있다면 `~/.bashrc`에 `ANTHROPIC_API_KEY`가 있으면 구독 대신 API 요금으로 청구됩니다. 구독으로 쓰려면 그 줄을 지우고 `unset ANTHROPIC_API_KEY` 후 다시 실행합니다.

✓ **성공하면** 마지막 명령이 `OK`를 출력합니다.

2.5 Antigravity CLI

이 단계에서 하는 일 scholar가 논문 웹 페이지를 읽을 때 쓰는 도구를 설치하고 Google 계정으로 로그인합니다.

```
curl -fsSL https://antigravity.google/cli/install.sh | bash
source ~/.bashrc
agy --version
```

공식 설치 스크립트가 agy 파일 하나를 ~/.local/bin/에 넣습니다. agy: command not found가 나오면 2.3절 팁처럼 PATH를 확인합니다.

이어서 agy를 한 번 실행해 Google 계정으로 로그인한 뒤 나옵니다. SSH로 접속해 브라우저가 열리지 않으면, 화면에 나온 주소를 내 PC 브라우저에 붙여 넣어 로그인합니다.

```
agy
```

로그인했으면 동작을 확인합니다.

```
agy -p "Reply with OK" --output-format json --print-timeout 2m
```

```
{ ... "status": "SUCCESS", ... }
```

읽기 권한(어떤 사이트를 읽어도 되는지)은 6장에서 한 번에 설정합니다.

✓ **성공하면** 출력 JSON의 "status"가 "SUCCESS"입니다.

2.6 연구실 설정 묶음 가져오기

이 단계에서 하는 일 직원들의 역할·규칙과 설치 도우미 스크립트가 든 hermes-team 폴더를 정해진 위치에 받습니다.

위치는 반드시 ~/Projects/hermes-team입니다. 스크립트들이 이 경로를 기준으로 동작합니다.

```
mkdir -p ~/Projects && cd ~/Projects
git clone https://github.com/<저장소>.git hermes-team
ls ~/Projects/hermes-team
```

```
docs  extras  INSTALL.md  install_souls.sh  LOCAL_MODEL.md  newproject.py  README.md
set_name.sh  SETUP.md  souls  template  tools
```

GitHub 대신 파일로 받았다면 hermes-team-share.tar를 받았다면 tar xf hermes-team-share.tar -C ~/Projects로 풀고 폴더 이름을 hermes-team으로 바꿉니다 (mv ~/Projects/hermes-team-share ~/Projects/hermes-team).

2.6.1 연구자 이름 정하기

직원들은 나를 이름으로 부르고 승인을 요청합니다. 원본 규칙 파일에는 이름 자리가 {{연구자}}로 비어 있고, 아래 명령이 내 이름으로 채웁니다.

```
cd ~/Projects/hermes-team
bash set_name.sh <내이름>        # 예: minji 또는 김민지 (공백 없이)
```

이 명령은 **내 컴퓨터 전용 설정 파일** 세 개(team.json, souls/_team_ids.md, projects.json)를 예시에서 만들고, commit할 때 개인 정보가 섞이지 않게 막는 검사를 켭니다. 이 세 파일은 git에 올라가지 않습니다. 나중에 다시 실행해도 됩니다.

> ✓ **성공하면** 다음: INSTALL.md 를 처음부터 따라 한다.가 출력되고, ls team.json souls/_team_ids.md projects.json에 세 파일이 모두 보입니다.

CHAPTER 3

Discord 서버 준비

3.1 개발자 모드 켜기

이 단계에서 하는 일 Discord에서 서버 · 채널 · 사용자의 ID를 복사할 수 있게 합니다.

Discord 앱(PC 권장) → 왼쪽 아래 **사용자 설정(톱니바퀴)** → **고급** → **개발자 모드** 켜기.

이제 서버, 채널, 사용자를 **우클릭**하면 메뉴 맨 아래에 **ID 복사**가 나옵니다.

3.2 서버와 채널 만들기

연구실 전용 서버가 없다면 왼쪽 서버 목록의 **+** → **직접 만들기** → **나와 친구들을 위한 서버**로 만듭니다. 그 서버 안에 다음을 만듭니다.

만들 것	이름	용도
텍스트 채널	`#hermes`	**로비.** 전체 상황, 새 프로젝트 요청. PI가 멘션 없이 답합니다
카테고리	`Projects`	PI가 프로젝트 채널을 **자동으로** 이 안에 만듭니다
텍스트 채널 (선택)	`#sandbox`	시험용

채널은 채널 목록의 **+** 버튼, 카테고리는 서버 이름 클릭 → **카테고리 만들기**로 만듭니다.

3.3 ID 네 개 복사하기

아래 네 곳을 우클릭 → **ID 복사**로 복사해 기록표에 붙여 넣습니다.

우클릭할 곳	기록표 칸
왼쪽 목록의 서버 아이콘	서버(guild) ID
`#hermes` 채널	로비 채널 ID
`Projects` 카테고리 이름	Projects 카테고리 ID
채팅창의 내 프로필(이름)	내 사용자 ID

✓ **성공하면** 기록표의 [Discord] 네 칸이 모두 18~19자리 숫자로 채워졌습니다.

CHAPTER 4

직원 6명 만들기

직원 한 명은 **Discord 봇 하나**와 **Hermes profile 하나**로 이루어집니다.

직원	Discord 봇 이름	Hermes profile
PI	`hermes-researcher`	`default` (기본 profile)
coder	`coder`	`coder`
analyst	`analyst`	`analyst`
reviewer	`reviewer`	`reviewer`
writer	`writer`	`writer`
scholar	`scholar`	`scholar`

4.1 Discord 봇 6개 만들기

이 단계에서 하는 일 Discord 개발자 페이지에서 봇을 만들고, 토큰을 받고, 내 서버에 초대합니다. 아래 1~5를 **봇마다 반복**합니다 (6번).

1. 브라우저에서 `https://discord.com/developers/applications`를 열고 Discord 계정으로 로그인합니다. 오른쪽 위 **New Application** → 이름 입력(위 표의 봇 이름) → 약관 동의 체크 → **Create**.
2. 왼쪽 **General Information**에서 **Application ID**를 **Copy** → 기록표의 그 봇 줄 "Application ID" 칸.
3. 왼쪽 **Bot**을 누릅니다.
   - **Reset Token** → 확인 → 나온 토큰을 **Copy** → 기록표 "토큰" 칸. **이 화면을 벗어나면 다시 볼 수 없습니다.**
   - 아래로 내려 **Privileged Gateway Intents**의 **Message Content Intent**를 켭니다 → 아래쪽 **Save Changes**.
4. 서버에 초대합니다. 브라우저 주소창에 아래 주소를 붙여 넣되, 끝의 `APP_ID`를 2번에서 복사한 값으로 바꿉니다.

   ```
   https://discord.com/oauth2/authorize?scope=bot+applications.commands&permissions=274878024768&client_id=APP_ID
   ```

   승인 화면 위쪽에 나온 봇 이름이 **지금 만드는 봇인지** 확인 → 서버 선택에서 내 연구실 서버 → **계속하기** → **승인**.
5. Discord 앱으로 돌아와 서버 오른쪽 멤버 목록에서 방금 들어온 봇을 우클릭 → **ID 복사** → 기록표 "봇 ID" 칸.

Message Content Intent를 꼭 켜세요 꺼져 있으면 봇이 메시지를 읽지 못해 게이트웨이가 접속에 실패합니다. 가장 흔한 실수입니다.

Application ID와 봇 ID 둘은 보통 같은 숫자입니다. 다르게 나와도 괜찮습니다. 기록표에는 각각 적어 둡니다. 나중에 쓰는 것은 **봇 ID**(5번)입니다.

✓ **성공하면** 서버 멤버 목록에 봇 6개(`hermes-researcher`, `coder`, `analyst`, `reviewer`, `writer`, `scholar`)가 오프라인으로 보이고, 기록표의 [봇] 표가 모두 채워졌습니다. 봇은 게이트웨이를 켜는 6.2절에서 온라인이 됩니다.

4.2 PI 설정

이 단계에서 하는 일 PI(기본 profile)를 완성합니다. 주 모델은 로컬 모델, 예비 모델은 Claude로 정하고 PI 봇 토큰을 넣습니다.

이 절은 **로컬 모델 서버가 이미 돌고 있다**는 전제에서 시작합니다. 아직 설치하지 않았다면 `LOCAL_MODEL.md`(또는 `LOCAL_MODEL.pdf`)를 먼저 따라 하고 돌아옵니다.

PI를 먼저 완성하는 이유는 4.3절에서 나머지 5명을 **PI를 복제해** 만들기 때문입니다. 여기서 정한 모델과 예비 모델이 그대로 복사되므로, 다섯 명에게는 봇 토큰을 넣고 analyst·reviewer만 ChatGPT로 바꾸면 됩니다.

4.2.1 로컬 모델 서버 확인

포트는 모델 서버를 설치한 방식에 따라 다릅니다.

- **개인 데스크탑**: `8080`
- **공용 서버**: 관리자에게 배정받은 포트 (`8081`~`8085` 중 하나)

포트 번호를 `~/.bashrc`에 `LLAMA_PORT`라는 이름으로 적어 둡니다. 그러면 터미널을 새로 열어도 `$LLAMA_PORT`라고 쓴 곳이 그 숫자로 바뀝니다. 첫 줄의 `8081` 자리에 **내 포트 숫자**를 넣어 실행합니다 (= 앞뒤에 띄어쓰기 없이).

```
echo 'export LLAMA_PORT=8081' >> ~/.bashrc
source ~/.bashrc
echo $LLAMA_PORT
curl -s http://127.0.0.1:$LLAMA_PORT/v1/models
```

```
{"object":"list","data":[{"id":"qwen3.6-35b-a3b", ... }]}
```

✓ **성공하면** 출력에 `qwen3.6-35b-a3b`가 보입니다.

아무것도 안 나오거나 `Connection refused` 모델 서버가 꺼져 있습니다. 데스크탑은 `systemctl --user status llama-server`, 공용 서버는 `systemctl status llama-qwen@<번호>`로 상태를 보고, `active (running)`이 아니면 `LOCAL_MODEL.md`의 해당 절을 다시 확인합니다.

포트가 바뀌면 `nano ~/.bashrc`로 맨 아래 `export LLAMA_PORT=` 줄의 숫자를 고치고, 5.2절의 "로컬 모델 포트가 바뀌었을 때" 명령으로 직원들의 주소도 고칩니다.

4.2.2 주 모델을 로컬 모델로

설정 파일(config.yaml)은 2천 줄이 넘어서, 손으로 열어 고치면 실수하기 쉽습니다. 그래서 파일을 직접 열지 않고 hermes config set <항목> <값> 명령으로 필요한 줄만 바꿉니다. 다른 줄과 주석은 건드리지 않습니다. 첫 줄은 원본 설정을 .before-local로 백업합니다.

```
cp "$(hermes config path)" "$(hermes config path).before-local"
hermes config set model.default qwen3.6-35b-a3b
hermes config set model.provider custom:local-qwen
hermes config set model.context_length 131072
hermes config set providers.local-qwen.base_url http://127.0.0.1:$LLAMA_PORT/v1
hermes config set providers.local-qwen.default_model qwen3.6-35b-a3b
```

```
✓ Set model.default = qwen3.6-35b-a3b in /ai/works/minji/.hermes/config.yaml
✓ Set model.provider = custom:local-qwen in ...
...
```

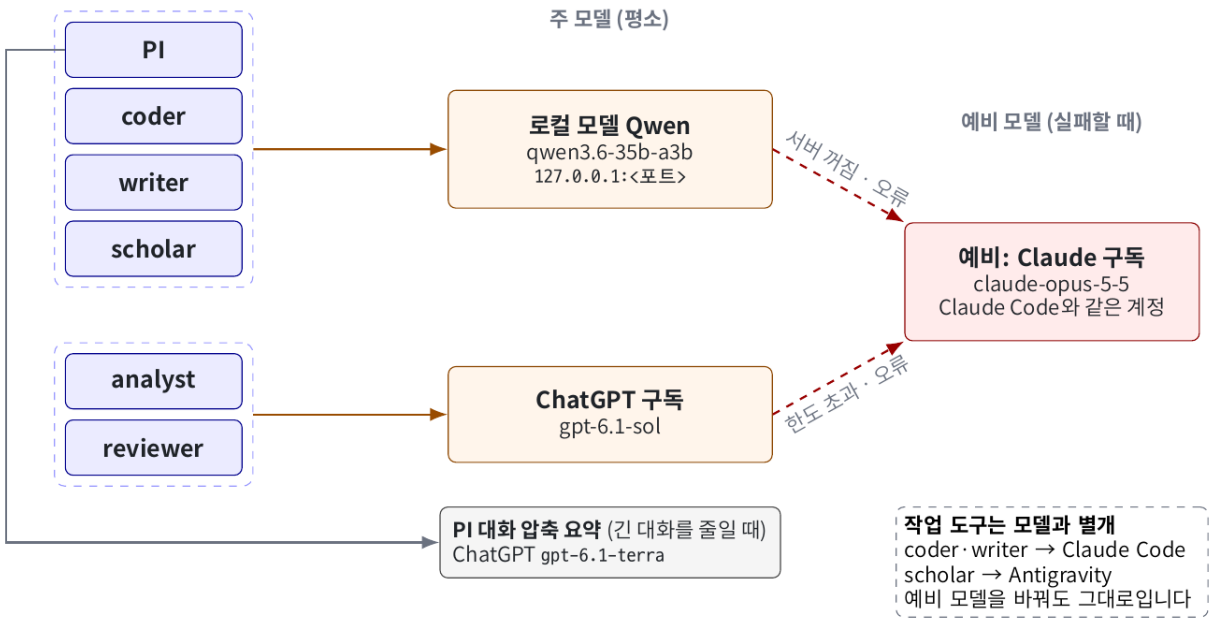
✓ **성공하면** 다섯 줄 모두 ✓ Set이 나옵니다.

항목	뜻
model.default	쓸 모델 이름. 모델 서버의 --alias와 똑같아야 합니다
model.provider	어디로 보낼지. custom:local-qwen은 아래 providers.local-qwen을 쓰라는 뜻입니다
model.context_length	한 번에 읽는 문맥 길이. 모델 서버의 **슬롯 하나당** 길이(131072)와 맞춥니다
providers.local-qwen.base_url	모델 서버 주소

되돌리기 처음 상태로 되돌리려면 백업을 덮어씁니다. 예: cp ~/.hermes/config.yaml.before-local ~/.hermes/config.yaml (직원은 ~/.hermes/profiles/<직원>/config.yaml.before-local)

4.2.3 예비 모델 넣기

주 모델이 실패하면 Claude가 대신 답하게 합니다.



예비 모델은 주 모델이 **꺼져 있거나, 오류를 내거나, 사용 한도에 걸렸을 때** 자동으로 쓰입니다.

- 로컬 모델 직원: 모델 서버가 꺼져 있거나 재시작 중일 때 Claude로 넘어갑니다.
- analyst · reviewer: ChatGPT 사용 한도에 걸리면 Claude로 넘어갑니다.
- Claude는 2.4절에서 로그인한 **Claude 구독**을 씁니다. 따로 로그인할 것은 없습니다.

Claude 구독으로 답하는 기능은 Hermes 기본 설치에 들어 있지 않고, Nous Research 공식 플러그인으로 따로 설치합니다. 2.4절에서 설치 · 로그인한 `claude` 명령을 통해 Claude 구독을 씁니다.

```
hermes plugins install claude-subscription-directsdk --enable
hermes plugins list | grep claude-subscription
```

✓ **성공하면** `claude-subscription-directsdk`가 `enabled`로 보입니다. 설치를 물으면 y로 답합니다.

설치되면 `hermes model`의 provider 목록에도 아래 줄이 생깁니다 (여기서 고르지는 않습니다. 고르면 PI의 주 모델이 Claude로 바뀝니다).

```
  (◯ Upstage (Solar API)
→ (◯ Claude Subscription DirectSDK (Experimental) (Claude Pro/Max subscription via your Claude Code login; Hermes owns tools)
  (◯ local-qwen (127.0.0.1:8081/v1) — qwen3.6-35b-a3b
```

Claude Subscription DirectSDK (Experimental): 예비 모델 설정에 쓰는 provider `claude-subscription-directsdk-experimental`

`Unknown provider 'claude-subscription-directsdk-experimental'`**이 나오면** 이 플러그인이 설치 · 활성화되지 않은 것입니다. 위 두 줄을 실행합니다. PI에 설치한 뒤 4.3절에서 복제하면 직원들에게도 같이 들어갑니다.

고장 난 예비 모델은 오히려 해롭습니다. 예비 모델까지 실패하면 진짜 실패로 세어지고, 같은 kanban 카드가 2번 실패하면 `blocked`가 됩니다. 그래서 넣기 전에 그 모델 하나만 따로 불러 봅니다.

```
hermes chat --provider claude-subscription-directsdk-experimental -m claude-opus-5-5 -q "OK라고만 답해"
```

✓ **성공하면** OK가 나옵니다. 오류가 나면 2.4절(Claude Code 로그인)부터 다시 확인하고, 통과할 때까지 다음으로 넘어가지 않습니다.

예비 모델 목록을 변수 FB에 담고 PI에 넣습니다. FB= 네 줄은 따옴표까지 **그대로, 통째로** 붙여 넣습니다. 작은따옴표가 닫힐 때까지 한 덩어리로 들어갑니다.

```
FB='[{"provider": "claude-subscription-directsdk-experimental",
      "model": "claude-opus-5-5",
      "base_url": "process://claude-subscription-directsdk-experimental",
      "api_mode": "chat_completions"}]'
hermes config set fallback_providers "$FB"
hermes fallback list
```

```
  Primary:   qwen3.6-35b-a3b  (via custom:local-qwen)
    1. claude-opus-5-5  (via claude-subscription-directsdk-experimental)  [process://...]
```

✓ **성공하면** Primary가 qwen3.6-35b-a3b이고, 1.에 claude-opus-5-5이 보입니다.

Claude Code 구독으로 쓰는 claude-opus-5-5의 문맥은 20만 토큰입니다. 로컬 모델(131072)보다 길어서, 긴 대화 중에 넘어가도 잘리지 않습니다. 이름 끝에 [1m]을 붙이면 100만 토큰판이 되지만, 이 연구실에는 필요 이상이고 긴 대화가 넘어오면 Claude 사용량을 그만큼 많이 씁니다.

Claude 사용량은 Claude Code와 같이 씁니다 coder·writer의 작성 도구(Claude Code), 아침 브리핑, 차단 감시도 같은 Claude 구독을 씁니다. 로컬 모델 서버가 오래 꺼져 있으면 로컬 모델 직원들이 모두 Claude로 생각하게 되어 사용량이 빨리 찹니다. 서버가 꺼진 것을 알면 바로 다시 켭니다.

4.2.4 Discord 연결

```
hermes gateway setup
```

1. 플랫폼 목록에서 **Discord**를 고릅니다.
2. 토큰을 물으면 기록표의 **PI (hermes-researcher)** 토큰을 붙여 넣습니다.
3. Allow yourself?에는 y.
4. Home channel은 그냥 Enter (6장에서 Discord에서 정합니다).
5. 마지막 Start the gateway now?와 이어지는 Start the gateway automatically on login/boot as a systemd service?에는 둘 다 n. 게이트웨이는 직원 6명을 모두 설정한 뒤 6.2절에서 서비스로 등록하고 한 번에 켭니다.

봇이 Discord에 접속되는지 미리 보고 싶다면 hermes gateway run을 실행하면 이 터미널에서 게이트웨이가 돌고, 멤버 목록에서 hermes-researcher가 온라인(초록 점)이 됩니다. 확인했으면 Ctrl + C로 끕니다.

4.2.5 시험

PI에게 말을 걸고, 그 요청을 **로컬 모델이 처리했는지**도 확인합니다. 모델 서버는 /slots 주소에서 지금까지 처리한 작업 번호(id_task)를 보여 줍니다. 대화 전과 후에 이 번호를 비교합니다.

```
curl -s http://127.0.0.1:$LLAMA_PORT/slots ¦ grep -o '"id_task":[0-9-]*'
hermes chat -q "OK라고만 답해"
curl -s http://127.0.0.1:$LLAMA_PORT/slots ¦ grep -o '"id_task":[0-9-]*'
```

```
"id_task":0
"id_task":39
OK
"id_task":53
"id_task":70
```

✓ **성공하면** PI가 `OK`라고 답하고, 두 번째 `id_task` 숫자 중 하나 이상이 첫 번째보다 커졌습니다. 로컬 모델이 답한 것입니다.

같은 모델 서버를 다른 사람과 같이 쓴다면 그 사람의 요청 때문에도 숫자가 늘 수 있습니다. 확실히 보려면 `hermes chat`이 도는 동안 다른 터미널에서 슬롯이 일하는 중(`"is_processing":true`)인지 봅니다.

```
curl -s http://127.0.0.1:$LLAMA_PORT/slots ¦ grep -o '"is_processing":[a-z]*'
```

예비 모델 때문에 고장이 가려질 수 있습니다 로컬 모델이 꺼져 있어도 Claude가 대신 답하므로 `OK`는 나옵니다. 그래서 `id_task` **숫자가 늘었는지**를 꼭 함께 확인합니다. 그대로라면 4.2.1절로 돌아갑니다.

이런 오류가 나면	원인과 해결
`connection refused` (예비도 실패)	모델 서버가 꺼져 있고 Claude 로그인도 안 된 상태입니다 → 4.2.1절, 2.4절
모델 이름 오류 (`model not found`)	모델 서버의 `--alias`와 `model.default`가 다릅니다
답은 오는데 `id_task`가 그대로	로컬 모델이 꺼져 Claude가 대신 답하는 중입니다 → 4.2.1절
답이 아주 느리거나 끊김	슬롯이 모자랍니다. `/slots`에서 모든 슬롯이 `"is_processing":true`인지 보고, 계속 그러면 관리자에게 알립니다

4.3 나머지 직원 5명 만들기

이 단계에서 하는 일 PI 설정을 복제해 profile 5개를 만들고, 각자 자기 봇 토큰을 넣고, analyst·reviewer만 ChatGPT로 바꿉니다.

4.3.1 profile 만들기

아래를 **통째로 복사해 한 번에** 붙여 넣습니다. 줄 끝의 \는 "다음 줄에 이어짐"이라는 뜻입니다. 따옴표 안에서도 똑같이 이어지므로, 설명 문장이 여러 줄이어도 한 문장으로 들어갑니다.

```
hermes profile create coder --clone --description "코드 구현·수정 담당. \
실제 코드 작성은 Claude Code(claude -p)에 맡기고, 결과를 테스트로 검증해 보고한다. \
연구 기록과 실험 명세는 수정하지 않는다."
hermes profile create analyst --clone --description "실험 실행과 결과 분석 담당. \
승인된 실험 명세만 실행하고, success_criteria 판정과 FINDINGS 수정안을 만든다. \
코드는 수정하지 않는다."
hermes profile create reviewer --clone --description "독립 검토 담당. \
코드·분석 결과·논문 초안·문헌 조사를 검토하고, \
GPT가 만든 판정은 Claude 읽기 전용 2차 의견을 받는다. 직접 수정하지 않는다."
hermes profile create writer --clone --description "논문·보고서 초안 담당. \
작성은 Claude Code에 docs/paper/ 쓰기만 허용해 맡기고, \
수치는 results와, 인용은 references.bib와 대조해 검증한다."
hermes profile create scholar --clone --description "문헌 조사 담당. \
관련 논문을 찾아 Antigravity로 읽고 근거 위치와 함께 정리하며, \
arXiv/DOI로 실제 존재를 검증해 docs/literature/와 references.bib에 남긴다."
hermes profile list
```

✓ **성공하면** `hermes profile list`에 `default`, `coder`, `analyst`, `reviewer`, `writer`, `scholar` 6개가 보입니다.

4.3.2 직원마다 자기 봇 토큰 넣기

이 단계를 건너뛰면 안 됩니다 `--clone`은 **PI의 봇 토큰까지 복사**합니다. 그대로 두면 6명이 같은 봇으로 접속하려다 한 명만 켜지고 나머지는 꺼집니다.

한 줄씩 실행하고, 매번 4.2.4절과 똑같이 답합니다: **Discord** → **그 직원의 토큰** → y → `Enter`.

```
hermes -p coder gateway setup       # coder 봇 토큰
hermes -p analyst gateway setup     # analyst 봇 토큰
hermes -p reviewer gateway setup    # reviewer 봇 토큰
hermes -p writer gateway setup      # writer 봇 토큰
hermes -p scholar gateway setup     # scholar 봇 토큰
```

`-p`의 뜻 `hermes -p coder ...`는 "coder profile로 실행"입니다. `-p`가 없으면 PI(`default`)입니다. 앞으로 자주 나옵니다.

4.3.3 analyst·reviewer는 ChatGPT로

복제했으므로 지금은 다섯 명 모두 PI처럼 로컬 모델이 주 모델입니다. 실험 분석과 독립 검토를 맡는 analyst·reviewer는 **로컬 모델과 다른 모델**이 하도록 ChatGPT로 바꿉니다. 같은 모델이 만든 실수를 같은 모델이 놓치는 일을 줄이기 위해서입니다.

먼저 ChatGPT 계정에 **한 번만** 로그인합니다. `-p` 없이(PI에서) 로그인하면, 자기 로그인이 없는 직원들은 PI의 로그인을 같이 씁니다. 화면에 나온 주소를 내 PC 브라우저에서 열고, 화면의 코드를 입력해 ChatGPT 계정으로 로그인합니다.

```
hermes auth add openai-codex
hermes -p analyst auth status openai-codex
hermes -p reviewer auth status openai-codex
```

> ✓ **성공하면** 두 직원 모두 `openai-codex: logged in`이 나옵니다.

`auth add`는 로그인만 하고 주 모델은 바꾸지 않으므로, PI는 로컬 모델 그대로입니다.

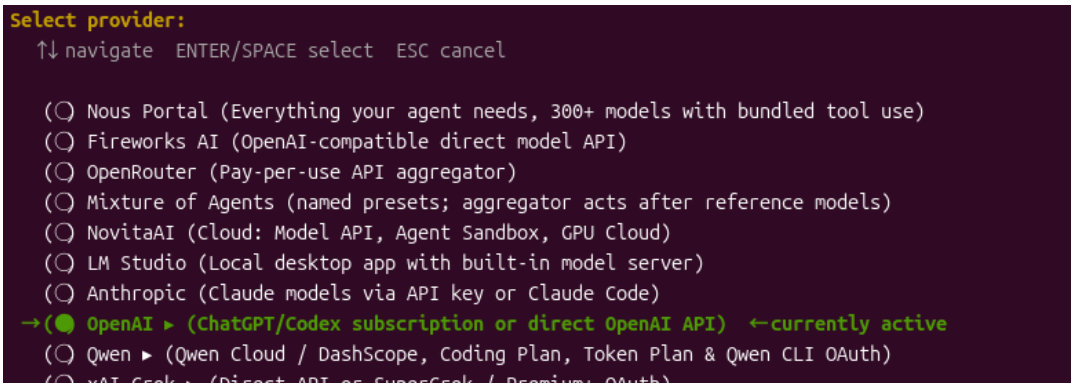
> **ChatGPT 계정이 두 개라면** 직원에게 따로 로그인하면 그 직원만 그 계정을 씁니다(자기 로그인이 PI 로그인보다 먼저입니다). 예: `hermes -p reviewer auth add openai-codex`를 실행해 브라우저에서 **두 번째 계정**으로 로그인하면, reviewer는 두 번째 계정, analyst는 PI의 첫 번째 계정을 씁니다. 사용 한도도 계정별로 따로 찹니다. 어느 계정이 들어갔는지는 `hermes -p reviewer auth list`로 봅니다.

그다음 주 모델을 바꿉니다. 예비 모델(Claude)은 복제할 때 들어간 그대로 남습니다.

```
hermes -p analyst config set model.provider openai-codex
hermes -p analyst config set model.default gpt-6.1-sol
hermes -p reviewer config set model.provider openai-codex
hermes -p reviewer config set model.default gpt-6.1-sol
```

> ✓ **성공하면** 네 줄 모두 `✓ Set`이 나옵니다.

> **메뉴로 골라도 됩니다** 위 네 줄 대신 `hermes -p analyst model` → **OpenAI (ChatGPT/Codex subscription ...)** → `gpt-6.1-sol`을 골라도 결과가 같습니다 (reviewer도 `-p reviewer`로). 예비 모델은 그대로 남습니다.



`hermes -p analyst model`의 provider 목록. 화살표 키로 **OpenAI**로 옮겨 `Enter`, 이어지는 메뉴에서 ChatGPT/Codex 구독과 모델을 고릅니다.

> **PI에서는 `hermes model`로 ChatGPT를 고르지 않습니다** `-p` 없이 `hermes model`에서 ChatGPT를 고르면 PI의 주 모델까지 ChatGPT로 바뀝니다. PI의 로그인은 위처럼 `auth add`로만 합니다. **Nous Portal** 로그인을 권하는 화면이 나오면 `Ctrl`+`C`로 나옵니다. 별도 유료 서비스입니다.

PI 대화 압축 요약도 ChatGPT로

대화가 길어지면 Hermes가 앞부분을 요약해 줄입니다. PI는 이 요약을 로컬 모델 대신 ChatGPT에 맡깁니다 (긴 글 요약은 큰 모델이 더 정확합니다). 위에서 이미 로그인했으므로 두 줄이면 됩니다. PI의 주 모델은 로컬 모델 그대로입니다.

```
hermes config set auxiliary.compression.provider openai-codex
hermes config set auxiliary.compression.model gpt-6.1-terra
```

4.3.4 확인

주 모델과 예비 모델을 한 명씩 봅니다.

```
hermes -p coder fallback list       # Primary: qwen3.6-35b-a3b
hermes -p writer fallback list      # Primary: qwen3.6-35b-a3b
hermes -p scholar fallback list     # Primary: qwen3.6-35b-a3b
hermes -p analyst fallback list     # Primary: gpt-6.1-sol
hermes -p reviewer fallback list    # Primary: gpt-6.1-sol
```

✓ **성공하면** 줄마다 `Primary`가 `#` 뒤에 적은 모델이고, `1.`에 `claude-opus-5-5`이 보입니다.

이제 다섯 명이 모두 모델에 닿는지 한 줄씩 확인합니다. coder·writer·scholar의 요청은 4.2.5절처럼 `/slots`의 `id_task`를 늘려야 합니다.

```
hermes -p coder chat -q "OK라고만 답해"
hermes -p analyst chat -q "OK라고만 답해"
hermes -p reviewer chat -q "OK라고만 답해"
hermes -p writer chat -q "OK라고만 답해"
hermes -p scholar chat -q "OK라고만 답해"
```

✓ **성공하면** 줄마다 `OK`가 나옵니다.

이 매뉴얼은 직원마다 한 줄씩 칩니다 앞으로 나오는 설정도 모두 `-p <직원>`만 바꿔 한 명씩 실행합니다. 그래서 나중에 직원을 새로 늘려도 같은 줄을 그 이름으로 한 번 더 치면 됩니다 (8.5절 “새 직원 추가하기”).

CHAPTER 5

다른 모델 고르기

4장의 구성(로컬 모델 4명 + ChatGPT 2명 + 예비 Claude Opus)은 기본값입니다. 직원별로 다른 모델을 쓰고 싶을 때 이 장을 봅니다. 처음 설치할 때는 건너뛰고 6장으로 가도 됩니다.

5.1 바꿀 때마다 지키는 순서

1. **단독 시험**: `hermes chat --provider <provider> -m <모델> -q "OK라고만 답해"`가 통과하는지 봅니다.
2. **바꾸기**: 아래 표의 명령을 실행합니다. `<직원>`은 `default`(PI), `coder`, `analyst`, `reviewer`, `writer`, `scholar` 중 하나입니다.
3. **적용**: `hermes gateway restart --all` 후, Discord에서 그 직원과 대화하던 채널에 `/reset`을 보냅니다 (열린 대화는 옛 모델을 붙잡고 있습니다).
4. **확인**: `hermes -p <직원> fallback list`에서 `Primary`와 예비 목록을 봅니다.

5.2 주 모델 바꾸기

하고 싶은 것	명령
ChatGPT의 다른 모델로	`hermes -p <직원> model` → **OpenAI (ChatGPT/Codex subscription …)** → 목록에서 모델 선택

로컬 모델 → ChatGPT로	메뉴: `hermes -p <직원> model` → **OpenAI (ChatGPT/Codex subscription …)** → 모델 선택
	또는 명령 (로그인이 안 되어 있으면 먼저 `hermes auth add openai-codex`):
	`hermes -p <직원> config set model.provider openai-codex`
	`hermes -p <직원> config set model.default gpt-6.1-sol`

ChatGPT → 로컬 모델로
(예: analyst도 로컬)
	`hermes -p <직원> config set model.provider custom:local-qwen`
	`hermes -p <직원> config set model.default qwen3.6-35b-a3b`
	(주소·문맥 길이는 복제 때 이미 들어 있습니다)

다른 로컬 모델로
(서버에 다른 모델을 띄웠을 때)
	새 모델의 `--alias`가 `<이름>`이면:
	`hermes -p <직원> config set model.default <이름>`
	`hermes -p <직원> config set providers.local-qwen.default_model <이름>`
	문맥 길이가 다르면 `model.context_length`도 슬롯당 길이로

로컬 모델 포트가 바뀌었을 때	`hermes -p <직원> config set providers.local-qwen.base_url http://127.0.0.1:<새포트>/v1`
처음 상태로	`.before-local` 백업을 덮어씁니다 (4.2.2절 팁)

GPU 없이 먼저 시작하고 싶다면 로컬 모델 서버가 아직 없거나 점검 중이면, 4명을 "로컬 모델 → ChatGPT로" 명령으로 바꿔 6명 모두 ChatGPT로 돌릴 수 있습니다. ChatGPT 사용량이 그만큼 늘어 한도에 더 자주 걸립니다. 서버가 준비되면 "ChatGPT → 로컬 모델로"로 되돌립니다.

5.3 예비 모델 바꾸기

4.2.3절의 `FB='...'` 줄만 아래 중 하나로 바꿔 실행하고, 이어서 바꿀 직원마다 `hermes -p <직원> config set fallback_providers "$FB"`를 한 줄씩 실행합니다.

선택	FB 값과 특징
Claude Opus (기본)	`FB='[{"provider": "claude-subscription-directsdk-experimental", "model": "claude-opus-5-5", "base_url": "process://claude-subscription-directsdk-experimental", "api_mode": "chat_completions"}]'` 가장 똑똑하고 문맥이 깁니다. Claude 사용량을 많이 씁니다
Claude Sonnet	`FB='[{"provider": "claude-subscription-directsdk-experimental", "model": "claude-sonnet-5-5", "base_url": "process://claude-subscription-directsdk-experimental", "api_mode": "chat_completions"}]'` 더 빠르고 사용량을 덜 씁니다. Claude Code와 사용량을 나눠 쓰는 것이 걱정될 때
ChatGPT	`FB='[{"provider": "openai-codex", "model": "gpt-6.1-sol"}]'` 로컬 모델 직원 4명에게만 의미가 있습니다. analyst·reviewer는 주 모델과 같은 계정이라, 한도에 걸리면 예비도 같이 막힙니다
두 단계 (Claude → ChatGPT)	`FB='[{"provider": "claude-subscription-directsdk-experimental", "model": "claude-opus-5-5", "base_url": "process://claude-subscription-directsdk-experimental", "api_mode": "chat_completions"}, {"provider": "openai-codex", "model": "gpt-6.1-sol"}]'` 목록 순서대로 시도합니다. Claude까지 실패하면 ChatGPT
예비 없음	`FB='[]'` ChatGPT 한도에 걸린 kanban 카드는 실패로 세지 않고 기다렸다가 한도가 풀리면 다시 시작합니다. 대신 로컬 모델 서버가 꺼지면 4명의 일이 실패하고, 2번 실패한 kanban 카드는 `blocked`가 됩니다

메뉴로 고르고 싶다면 `hermes -p <직원> fallback add`는 `hermes model`과 같은 메뉴로 예비 모델을 목록 끝에 하나 추가합니다. Claude는 **Claude Subscription DirectSDK (Experimental)**을 고릅니다 (4.2.3절 그림). `fallback remove`는 하나를 골라 지우고, `fallback clear`는 모두 지웁니다 (y로 확인).

5.4 그 밖에 바꿀 수 있는 것

무엇	방법
PI 대화 압축 요약 모델	`hermes config set auxiliary.compression.model <모델>` (ChatGPT 모델 이름)
생각 깊이 (reasoning)	`team.json`의 `"reasoning"`에서 직원별 `low/medium/high`를 고친 뒤 `python3 tools/lab.py reasoning`
지금 설정 보기	`hermes -p <직원> config get model.default`, `hermes -p <직원> fallback list`

CHAPTER 6

팀으로 묶기

지금은 직원 6명이 각자 따로 있습니다. 이 장에서 서로 멘션하며 협업하고, 역할 규칙을 지키고, 프로젝트 채널을 다룰 수 있게 묶습니다. 대부분은 설치 도우미 스크립트가 해 주므로 **ID만 정확히 적으면** 됩니다.

6.1 팀 설정 파일 채우기

이 단계에서 하는 일 기록표의 ID를 두 파일에 옮겨 적습니다. 이후 모든 스크립트가 이 값을 읽습니다.

6.1.1 team.json

```
cd ~/Projects/hermes-team
nano team.json
```

`guild_id`, `category_id`, `lobby_channels` 세 곳의 숫자를 기록표 값으로 바꿉니다. 나머지는 그대로 둡니다.

/Projects/hermes-team/team.json (예)

```
{
  "guild_id": "11111111111111111",          ← 서버(guild) ID
  "category_id": "22222222222222222",       ← Projects 카테고리 ID
  "projects_root": "~/Projects",
  "profiles": ["default", "coder", "analyst", "reviewer", "writer", "scholar"],
  "lobby_channels": ["33333333333333333"],  ← 로비 채널(#hermes) ID
  "reasoning": {
    "default": "medium", "coder": "low", "analyst": "medium",
    "reviewer": "medium", "writer": "low", "scholar": "low"
  }
}
```

화살표와 설명은 적지 않습니다 위 예의 ← …는 설명입니다. 실제 파일에 넣으면 JSON이 깨집니다. 숫자는 따옴표 " " 안에 넣고, 쉼표와 괄호는 그대로 둡니다.

6.1.2 souls/_team_ids.md

```
nano souls/_team_ids.md
```

맨 위 일곱 줄의 대문자 자리(`RESEARCHER_USER_ID`, `PI_BOT_ID` …)를 기록표의 **내 사용자 ID**와 **봇 ID**로 바꿉니다. <@ >

기호는 그대로 둡니다. 연구자 이름 자리는 set_name.sh가 이미 채웠습니다.

/Projects/hermes-team/souls/_team_ids.md (위쪽)

```
## 팀 Discord ID (멘션할 때 이 형식을 그대로 쓴다)
- 연구자 minji: `<@444444444444444444>`
- PI: `<@555555555555555555>`
- coder: `<@666666666666666666>`
- analyst: `<@777777777777777777>`
- reviewer: `<@888888888888888888>`
- writer: `<@999999999999999999>`
- scholar: `<@000000000000000000>`
```

Application ID가 아니라 봇 ID 여기에는 4.1절 5번에서 멤버 목록을 우클릭해 복사한 **봇 ID**를 넣습니다.

6.2 게이트웨이를 서비스로 켜기

이 단계에서 하는 일 직원 6명을 한꺼번에 맡는 게이트웨이를 systemd 서비스로 등록하고 켭니다. 이후의 restart는 모두 이 서비스를 다시 시작합니다.

혹시 hermes gateway run으로 띄워 둔 게이트웨이가 있으면 먼저 끕니다(그 창에서 Ctrl+C). 같은 봇이 두 번 뜨면 나중에 켠 쪽이 연결에 실패합니다.

```
hermes gateway install
hermes gateway status
```

질문 두 개(Start the gateway now ...?, ... automatically on login/boot with systemd?)에는 둘 다 y.

```
● hermes-gateway.service - Hermes Agent gateway
     Active: active (running) since ...
```

✓ **성공하면** hermes-gateway.service와 active (running)이 보입니다. Discord 멤버 목록에서 봇 6개가 온라인(초록 점)이 됩니다.

Failed to enable unit: ... does not exist가 나오면 gnode1이 내 홈을 아직 /home/<계정>으로 보고 있는 것입니다. 6.9.1절을 따릅니다.

6.3 한 번에 적용하기

이 단계에서 하는 일 위 두 파일을 읽어 6명의 협업 설정, 생각 깊이, 채널 연결, 역할 규칙(SOUL)을 한꺼번에 설치합니다.

```
cd ~/Projects/hermes-team
python3 tools/lab.py env          # 6명 .env: 봇끼리 멘션 허용, 허용 사용자 = 나 + 봇 6개
python3 tools/lab.py reasoning    # 생각 깊이: PI·analyst·reviewer medium / coder·writer·scholar low
python3 newproject.py sync        # 6명 config.yaml: 로비·프로젝트 채널 연결 (PI만 멘션 없이 응답)
bash install_souls.sh             # 6명 SOUL(역할·규칙) 설치 + 게이트웨이 재시작
```

- `lab.py env`는 각 직원의 `.env`에 협업 설정(`DISCORD_ALLOW_BOTS=mentions` 등)과 허용 사용자 7명을 적습니다. 원본은 `.env.bak`으로 남습니다.
- `install_souls.sh`가 **"주의: 채우지 않은 ID"**를 출력하면 6.1절로 돌아가 빠진 ID를 채우고 다시 실행합니다.

✓ **성공하면** 네 명령이 오류 없이 끝나고, `install_souls.sh`가 ID 경고 없이 게이트웨이를 재시작합니다.

게이트웨이가 이 창을 차지하면 (`Press Ctrl+C to stop`) 게이트웨이가 서비스로 등록되지 않은 것입니다(6.2절). `Ctrl` + `C`로 끄고 6.2절부터 다시 합니다. 등록이 실패하면 6.9.1절을 봅니다.

6.4 직원 봇의 슬래시 명령 등록 (처음 한 번)

이 단계에서 하는 일 `/reset` 같은 Discord 명령을 직원 봇 5개에도 등록합니다.

복제한 profile은 PI의 "명령 등록 완료" 기록까지 복사돼서, 그대로 두면 직원 봇에 명령이 생기지 않습니다. 한 번만 강제로 등록합니다. 아래를 붙여 넣습니다 (마지막에 30초 기다립니다).

```
echo 'DISCORD_COMMAND_SYNC_POLICY=bulk' >> ~/.hermes/profiles/coder/.env
echo 'DISCORD_COMMAND_SYNC_POLICY=bulk' >> ~/.hermes/profiles/analyst/.env
echo 'DISCORD_COMMAND_SYNC_POLICY=bulk' >> ~/.hermes/profiles/reviewer/.env
echo 'DISCORD_COMMAND_SYNC_POLICY=bulk' >> ~/.hermes/profiles/writer/.env
echo 'DISCORD_COMMAND_SYNC_POLICY=bulk' >> ~/.hermes/profiles/scholar/.env
hermes gateway restart --all
sleep 30
```

직원마다 등록 기록이 생겼는지 한 줄씩 봅니다. 숫자는 등록한 횟수입니다.

```
grep -c -i 'Synced.*slash' ~/.hermes/profiles/coder/logs/gateway.log
grep -c -i 'Synced.*slash' ~/.hermes/profiles/analyst/logs/gateway.log
grep -c -i 'Synced.*slash' ~/.hermes/profiles/reviewer/logs/gateway.log
grep -c -i 'Synced.*slash' ~/.hermes/profiles/writer/logs/gateway.log
grep -c -i 'Synced.*slash' ~/.hermes/profiles/scholar/logs/gateway.log
```

✓ **성공하면** 다섯 줄 모두 `1` 이상입니다.

`0`인 직원이 있으면 30초로 모자랐을 수 있습니다. 1분쯤 더 기다렸다가 그 줄만 다시 실행합니다. 그래도 `0`이면 그 직원의 `.env`에 강제 등록 줄이 들어갔는지(`grep SYNC ~/.hermes/profiles/<직원>/.env`) 보고, `hermes gateway restart --all` 후 다시 확인합니다.

등록이 끝났으니 강제 등록 줄을 다시 지웁니다. 남겨 두면 재시작할 때마다 명령을 새로 등록합니다.

```
sed -i '/^DISCORD_COMMAND_SYNC_POLICY=bulk$/d' ~/.hermes/profiles/coder/.env
sed -i '/^DISCORD_COMMAND_SYNC_POLICY=bulk$/d' ~/.hermes/profiles/analyst/.env
sed -i '/^DISCORD_COMMAND_SYNC_POLICY=bulk$/d' ~/.hermes/profiles/reviewer/.env
sed -i '/^DISCORD_COMMAND_SYNC_POLICY=bulk$/d' ~/.hermes/profiles/writer/.env
sed -i '/^DISCORD_COMMAND_SYNC_POLICY=bulk$/d' ~/.hermes/profiles/scholar/.env
hermes gateway restart --all
```

✓ **성공하면** Discord에서 Ctrl+R로 새로고침하고 채팅창에 /를 입력하면 봇마다 명령 목록이 보입니다.

6.5 PI에게 채널 만들 권한 주기

이 단계에서 하는 일 PI가 새 프로젝트 채널을 Projects 카테고리 안에 만들 수 있게 합니다.

Discord에서 Projects 카테고리 이름 우클릭 → **카테고리 편집** → **권한** → **멤버 또는 역할 추가** → hermes-researcher → **채널 관리**를 초록 체크(허용)로 → **변경 사항 저장**.

서버 전체가 아니라 카테고리에만 서버 전체 관리 권한을 주지 않고 이 카테고리 안에서만 허용하는 것이 안전합니다.

6.6 Antigravity 읽기 권한

이 단계에서 하는 일 scholar가 학술 사이트를 읽을 수 있게 허용하고, 페이지 조작은 막습니다.

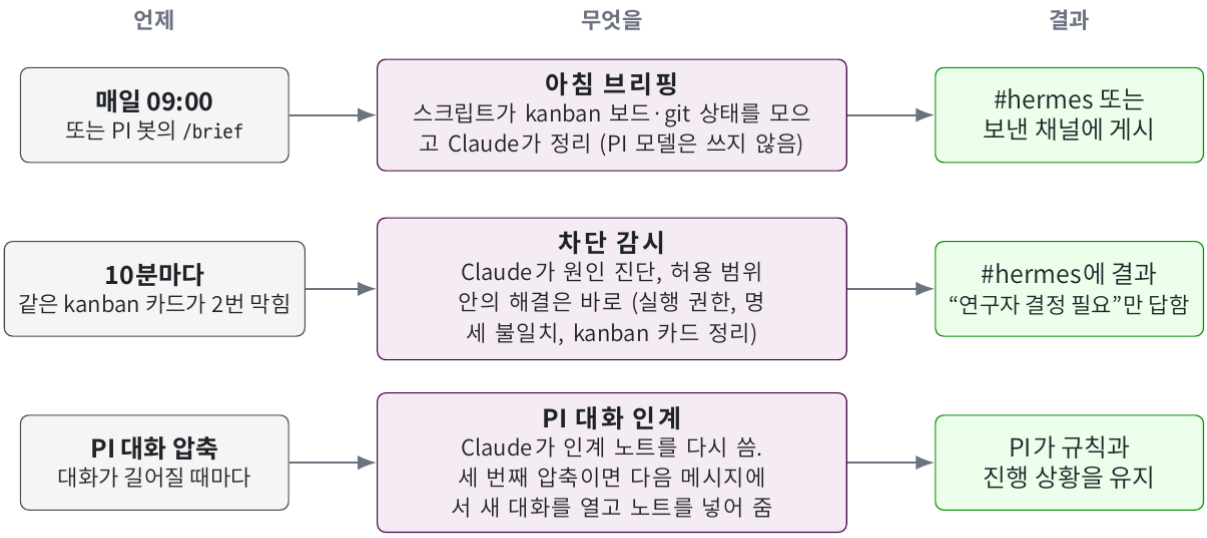
agy -p 같은 무인 실행은 허용되지 않은 사이트 읽기를 **오류 없이 조용히 거부**합니다. 그래서 미리 허용 목록을 넣어 둡니다.

```
python3 ~/Projects/hermes-team/tools/lab.py agy-perms
```

학술 도메인 20개 읽기 허용과 페이지 조작 거부 규칙이 ~/.gemini/antigravity-cli/settings.json에 합쳐집니다. 기존 규칙은 유지되고 원본은 settings.json.bak으로 남습니다.

6.7 자동 장치 설치 (아침 브리핑, 차단 감시, PI 인계)

이 단계에서 하는 일 사람이 챙기지 않아도 돌아가는 자동 장치 세 가지를 설치합니다.



설치: `extras/install_extras.sh` 매 턴 PI 규칙 리마인더(`souls/pi-rules.md`)도 같은 플러그인이 붙입니다

장치	하는 일
아침 브리핑	매일 09:00 프로젝트 상태를 Claude가 정리해 `#hermes`에 올립니다. Discord에서 PI 봇의 `/brief`로 언제든 받을 수도 있습니다
차단 감시	10분마다 확인해, 같은 일이 2번 이상 막히면 Claude가 원인을 진단하고 허용 범위 안에서 바로 고친 뒤 `#hermes`로 알립니다
PI 대화 인계	PI 대화가 길어지면 규칙을 놓치므로, 인계 노트를 써 두었다가 새 대화를 열 때 자동으로 넣어 줍니다

```
bash ~/Projects/hermes-team/extras/install_extras.sh
bash ~/Projects/hermes-team/install_souls.sh      # 게이트웨이 재시작 → hook·플러그인 적용
hermes plugins list | grep lab-handoff
hermes cron status
hermes cron list
```

✓ **성공하면** `lab-handoff`가 `enabled`이고, `hermes cron status`에 `Gateway is running`과 `2 active job(s)`가 보입니다. `hermes cron list`에서 `morning-brief`의 `Next run`이 **다음 09:00 (+09:00)**입니다.

`cron status`의 Next run이 10분 안쪽이면 정상입니다. 두 작업 중 가장 가까운 것을 보여 주는데, 차단 감시가 10분마다 돌기 때문입니다.

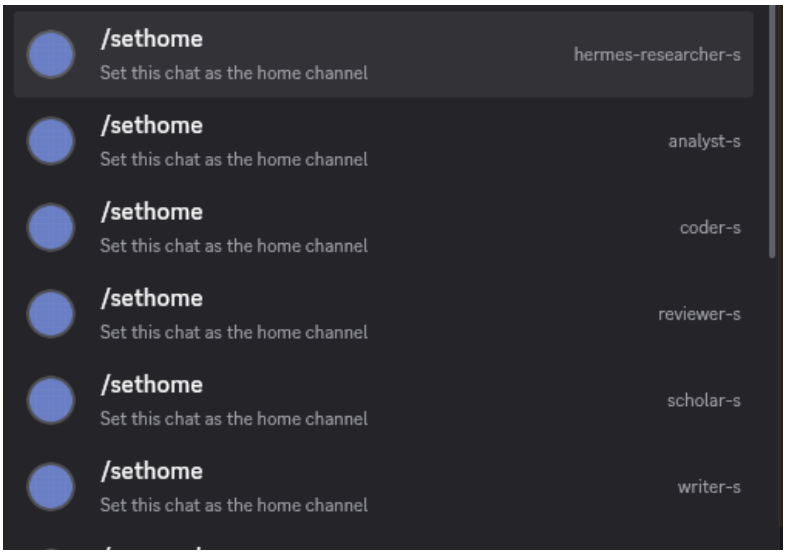
브리핑 시각이 9시간 어긋나면 서버 시간대가 UTC인 경우입니다 (`date`로 확인). 공용 서버의 시간대는 바꾸지 말고, 브리핑 cron을 UTC 기준 `"0 0 * * *"`(한국 시각 09:00)로 다시 만듭니다.

여러 번 실행해도 됩니다 `install_extras.sh`는 이미 있는 것은 건너뜁니다. `extras/`의 파일을 고친 뒤에도 다시 실행합니다.

6.8 PI 홈 채널 정하기

이 단계에서 하는 일 PI가 브리핑과 알림을 보낼 기본 채널을 #hermes로 정합니다.

Discord #hermes 채널에서 /sethome을 입력합니다. 봇 6개가 모두 같은 명령을 갖고 있어서 목록에 /sethome이 여러 줄 뜹니다. 아이콘이 모두 같으므로 **오른쪽 끝의 봇 이름**을 보고, **hermes-researcher**(PI) 줄을 골라 보냅니다.



맨 위 줄처럼 오른쪽에 **hermes-researcher**가 적힌 항목을 고릅니다. 다른 줄을 고르면 그 직원의 홈 채널이 바뀝니다.

봇마다 아이콘을 달아 두면 헷갈리지 않습니다 Discord 개발자 페이지(4.1절)에서 봇을 고른 뒤 두 곳에 그림을 올립니다. **General Information** → **App Icon**은 위 그림 같은 **명령 목록**의 아이콘, **Bot** → **Icon**은 **멤버 목록과 대화**에 보이는 봇 프로필 사진입니다. **Save Changes**를 누르면 바로 바뀌고, 서버에 다시 초대할 필요는 없습니다. 안 바뀌어 보이면 Discord 앱에서 `Ctrl` + `R`로 새로고침합니다.

이어서 터미널에서 브리핑을 한 번 시험합니다.

```
hermes cron run morning-brief
```

✓ **성공하면** 1~3분 뒤 #hermes에 브리핑이 올라옵니다 (아직 프로젝트가 없으므로 내용은 짧습니다).

6.9 SSH를 끊어도 계속 도는지 확인

이 단계에서 하는 일 로그인하지 않아도, 서버를 다시 켜도 게이트웨이가 저절로 도는지 확인합니다.

```
loginctl show-user $USER -p Linger
hermes gateway status
```

linger는 "내가 SSH 접속을 끊어도 내 서비스를 계속 돌려라"라는 설정입니다. 이게 없으면 SSH를 끊는 순간 직원들도 같이 꺼집니다.

```
Linger=yes
● hermes-gateway.service - Hermes Agent gateway
     Active: active (running) since Thu 2026-11-06 14:02:11 KST; 3h ago
```

✓ **성공하면** `Linger=yes`이고 `active (running)`이면 SSH를 끊어도, 서버를 다시 켜도 직원들이 계속 일합니다.

확실히 해 보려면 SSH를 끊었다가 다시 접속해 `hermes gateway status`를 봅니다. `since` 시각이 그대로면 그동안 한 번도 꺼지지 않은 것입니다. 게이트웨이를 끄고 켤 때는 `hermes gateway stop` / `hermes gateway start`를 씁니다.

6.9.1 서비스가 등록되지 않을 때 (홈 경로 문제)

`hermes gateway install`이 아래처럼 `Failed to enable unit: ... does not exist`로 끝나면, systemd가 내 홈을 `/home/<계정>`으로 보고 있는 것입니다(1.7.3절의 확인이 통과하지 않은 경우). 서비스 파일은 `/ai/works/<계정>/.config/systemd/user/`에 만들어졌는데, systemd는 `/home/<계정>` 쪽에서 찾기 때문입니다.

```
(base) member@gnode1:~/Projects/hermes-team$ hermes gateway status
✓ Gateway is running (PID: 178507)
  (Running manually, not as a system service)

To install as a service:
  hermes gateway install
  sudo hermes gateway install --system
(base) member@gnode1:~/Projects/hermes-team$ hermes gateway install
Start the gateway now after installing the service? [Y/n]: n
Start the gateway automatically on login/boot with systemd? [Y/n]: y
Installing user systemd service to: /ai/works/member/.config/systemd/user/hermes-gateway-6fd7b26e.service
Failed to enable unit: Unit hermes-gateway-6fd7b26e.service does not exist
✗ Could not start the gateway service; systemd reported an error.
  See why with `hermes gateway status --deep` or `journalctl --user -u hermes-gateway --since "5 min ago"`.
  To reinstall the service run `hermes gateway install --force`.
  Details: Command '['systemctl', '--user', 'enable', 'hermes-gateway-6fd7b26e']' returned non-zero exit status 1.
(base) member@gnode1:~/Projects/hermes-team$ systemctl --user show-environment | grep HOME
HOME=/home/member
(base) member@gnode1:~/Projects/hermes-team$ getent passwd member
member:x:1018:1018:,,,:/home/member:/bin/bash
(base) member@gnode1:~/Projects/hermes-team$
```

gnode1에서 실제로 나온 화면. 아래 두 줄이 `/home/<계정>`을 가리키는 것이 원인입니다.

① **원인 확인.** 아래 두 줄이 모두 `/home/<계정>`을 보여 주면 이 문제입니다.

```
systemctl --user show-environment | grep HOME
getent passwd $USER
```

② **관리자에게 부탁.** 위 화면을 캡처해 보내며 이렇게 부탁합니다. 계정을 새로 만드는 것이 아니라, **같은 UID 그대로 gnode1에서만 홈 경로를 바꾸는** 것입니다.

관리자에게 보낼 메시지

gnode1에서 systemd 사용자 서비스가 제 홈을 /home/<계정>으로 보고 있어 서비스 등록이 안 됩니다(캡처 첨부). gnode1의 `/etc/passwd`에 아래 한 줄을 추가해 주실 수 있을까요? UID·GID는 `getent passwd <계정>` 결과와 같고 홈만 다릅니다.

`<계정>:x:<UID>:<GID>:,,,:/ai/works/<계정>:/bin/bash`

nsswitch.conf가 files를 NIS보다 먼저 보므로 다른 노드에는 영향이 없습니다. 추가 후 `systemctl restart user@<UID>`도 부탁드립니다. (Linger가 no라면 `loginctl enable-linger <계정>`도 함께)

③ **고쳐질 때까지는 6장을 멈춥니다.** 게이트웨이는 서비스로 등록한 뒤에 켭니다. 관리자가 설정해 주면 아래 ④부터 이어서 합니다.

④ **관리자가 고쳐 준 뒤: 사용자 systemd 다시 띄우기.** 먼저 계정 정보가 바뀌었는지 봅니다.

```
getent passwd $USER
```

✓ **성공하면** `/ai/works/<계정>`이 보입니다. 아직 `/home/<계정>`이면 관리자 설정이 반영되지 않은 것이니 관리자에게 다시 확인합니다.

여기서 끝이 아닙니다. 내 사용자 systemd는 **예전 홈일 때 시작되어 그때 정한 경로에서만 서비스 파일을 찾고** 있고, Linger가 켜져 있어 SSH를 끊어도 꺼지지 않습니다. `show-environment`의 `HOME`이 새 값으로 보여도, `daemon-reload`를 해도 찾는 경로는 그대로입니다. 그래서 systemd를 한 번 내렸다가 새로 띄웁니다. sudo는 필요 없습니다.

```
systemctl --user exit
```

그다음 **SSH를 모두 끊고 다시 접속**해서, systemd가 실제로 찾는 경로(`UnitPath`)에 새 홈이 들어갔는지 확인합니다.

```
systemctl --user show -p UnitPath ¦ tr ' ' '\n' ¦ grep works
```

```
/ai/works/minji/.config/systemd/user.control
/ai/works/minji/.config/systemd/user
...
```

✓ **성공하면** `/ai/works/<계정>/.config/systemd/user`가 보입니다. 아무것도 안 나오면 관리자에게 `sudo systemctl restart user@<UID>`를 부탁합니다.

`systemd-analyze --user unit-paths`로 확인하면 안 됩니다 이 명령은 systemd에 묻지 않고 **지금 셸의 HOME**으로 경로를 계산하므로, systemd가 예전 경로를 보고 있어도 정상처럼 나옵니다. 꼭 `systemctl --user show -p UnitPath`로 봅니다.

⑤ **서비스로 등록.** 먼저 정리합니다.

- `hermes gateway run`으로 켜 둔 창이 있으면 `Ctrl`+`C`로 끕니다. 같은 봇이 두 번 뜨지 않도록 **먼저 끄는 것**이 중요합니다.
- 실패할 때 남은 예전 서비스 파일(`hermes-gateway-숫자·영문.service`)이 있으면 지웁니다. 남겨 두면 나중에 게이트웨이가 두 개 뜰 수 있습니다.

```
ls ~/.config/systemd/user/
rm -f ~/.config/systemd/user/hermes-gateway-*.service
```

이제 등록합니다. 질문 두 개(`Start the gateway now ...?`, `... automatically on login/boot ...?`)에는 둘 다 `y`.

```
hermes gateway install --force
hermes gateway status
python3 ~/Projects/hermes-team/tools/lab.py check
```

> ✓ **성공하면** `hermes gateway status`에 `hermes-gateway.service`와 `active (running)`이 보이고, `lab.py check`의 게이트웨이 줄이 OK입니다. 이제 SSH를 끊어도, 서버를 다시 켜도 직원들이 계속 일합니다. 끄고 켤 때는 `hermes gateway stop` / `start`를 씁니다. 6장을 진행하던 중이었다면 6.2절 다음부터 이어서 합니다.

CHAPTER 7

점검과 첫 대화

7.1 자동 점검

이 단계에서 하는 일 지금까지의 설치를 한 번에 확인합니다. 아무것도 바꾸지 않으니 언제 실행해도 안전합니다.

```
python3 ~/Projects/hermes-team/tools/lab.py check
```

도구, 팀 설정, 6명의 토큰 중복 여부, 협업 설정, SOUL 최신 여부, 채널 연결, 생각 깊이, 게이트웨이, 아침 브리핑, Antigravity 권한을 차례로 봅니다.

```
OK    hermes v0.22.5
OK    team.json: guild / category / lobby
...
READY — 모든 점검 통과
```

✓ **성공하면** 마지막 줄이 READY입니다.

WARN 줄이 있으면 그 줄에 고치는 명령이 함께 나옵니다. 그 명령을 실행하고 check를 다시 돌립니다.

7.2 Discord에서 시험하기

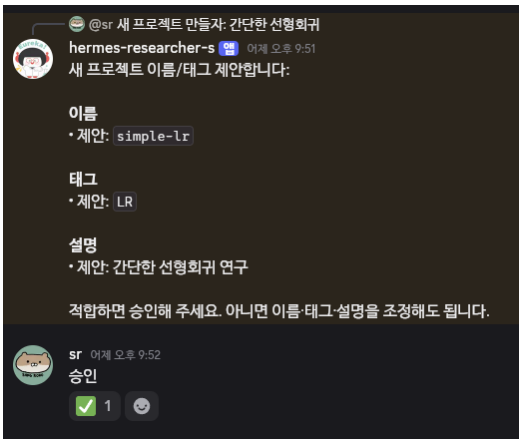
아래를 위에서부터 차례로 해 봅니다.

#	어디서	보낼 메시지	기대 결과
1	#hermes	넌 누구야? (멘션 없이)	PI만 답합니다
2	#hermes	@coder 넌 누구야?	coder만 답합니다. PI는 –만 보내거나 조용합니다
3	#hermes	analyst, reviewer, writer, scholar에게도 2번처럼	각자 자기 역할을 소개합니다
4	#hermes	새 프로젝트 만들자: 간단한 선형회귀	PI가 이름·태그를 제안 → 승인 → Projects에 새 채널, ~/Projects/<이름> 폴더, kanban 보드가 생기고 링크를 보고합니다. **보고 직후 게이트웨이가 15초 뒤 재시작됩니다**
5	새 프로젝트 채널 (**30초쯤 기다렸다가**)	상태 알려줘	첫 줄에 [태그], 표 형식 상태 보고. PI는 새 채널에서 먼저 말을 걸지 않으므로 내가 첫 메시지를 보냅니다

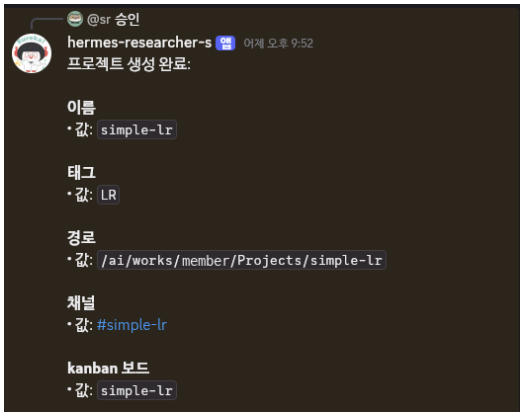
#	어디서	보낼 메시지	기대 결과
6	새 프로젝트 채널	데이터 생성 스크립트 하나 만들어줘	PI가 kanban 카드 생성 → coder가 Claude Code로 작성 → reviewer 검토 → PI가 `main`에 commit
7	새 프로젝트 채널	`@coder 기억해: 코드 스타일 – 타입 힌트 필수`	답 끝에 `기억함: ...`

> **멘션하는 법** `@`를 입력하고 자동완성 목록에서 **APP** 표시가 붙은 봇을 고릅니다. 같은 이름의 역할(role)을 고르면 봇이 반응하지 않습니다.

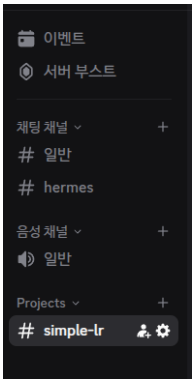
4번을 실제로 해 본 화면입니다. PI가 이름·태그·설명을 제안하면 승인(또는 좋아)으로 답하고, 바꾸고 싶으면 이름은 `lr-demo`로 처럼 고쳐 말합니다.



① 요청 → PI의 제안 → 승인



② 생성 완료 보고 (경로·채널·kanban 보드)



③ `Projects`에 생긴 채널

보고가 끝나고 15초쯤 뒤 게이트웨이가 다시 시작되며 봇들이 잠깐 오프라인이 됩니다. 30초쯤 기다렸다가 새 채널(여기서는 `#simple-lr`)에서 5번을 보냅니다.

6번의 진행 상황은 터미널에서도 볼 수 있습니다.

```
hermes kanban --board <프로젝트> list
```

7.3 반응이 없을 때

1. **재시작 중일 수 있습니다.** 30초~1분 기다렸다가 다시 보냅니다. 확인: `hermes gateway status`
2. 그래도 조용하면 그 채널에서 그 봇에게 `/reset`을 보냅니다 (/ 입력 → 그 봇의 명령 목록에서 `/reset`). 이전 대화가 옛 설정을 붙잡고 있을 때 생깁니다.
3. 그래도 안 되면 채널이 등록됐는지 확인하고 게이트웨이를 다시 켭니다.

```
python3 ~/Projects/hermes-team/newproject.py list
python3 ~/Projects/hermes-team/tools/lab.py check
hermes gateway restart --all
```

여기까지 왔다면 설치는 끝났습니다.
이제부터는 터미널 없이 Discord에서 일을 맡기면 됩니다. 다음 장은 매일 쓰는 법입니다.

7.4 예시: 분자 용해도 예측으로 연구 한 바퀴

이 단계에서 하는 일 `#simple-lr` 프로젝트에서 실제로 해 본 기록입니다. 아래 메시지를 같은 순서로 보내면 내 프로젝트에서도 가설 → 실험 → 결과 → 다음 단계를 한 바퀴 돌려 볼 수 있습니다.

주제. 분자가 물에 얼마나 녹는지(logS)를 분자 지표 4개로 선형회귀하는 고전 예제입니다. Delaney(2004)가 ESOL이라는 이름으로 발표한 방법이고, 그 데이터(분자 1,128개)는 MoleculeNet의 **ESOL** 데이터셋으로 공개되어 있습니다. 지표 4개는 logP(기름/물 분배 계수), 분자량, 회전 가능한 결합 수, 방향족 비율(AROM)입니다. CPU로 몇 분이면 끝나서 GPU가 필요 없습니다.

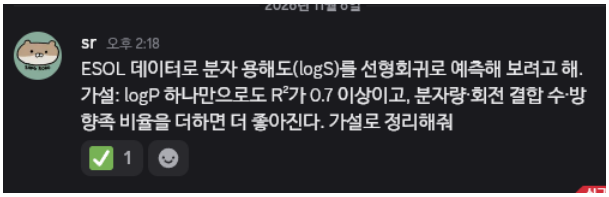
시작 전에 새 프로젝트를 하나 만듭니다(7.2절 4번처럼 `#hermes`에서 `새 프로젝트 만들자: 분자 용해도 선형회귀`). 아래 메시지는 모두 **그 프로젝트 채널**에 보냅니다. gnode1에서 인터넷이 막혀 있으면 데이터 내려받기에서 멈춥니다. 그때는 ESOL CSV를 직접 받아 `~/Projects/<프로젝트>/data/`에 넣고 "데이터는 `data/delaney.csv`에 있어"라고 알려 줍니다.

① 가설 세우기

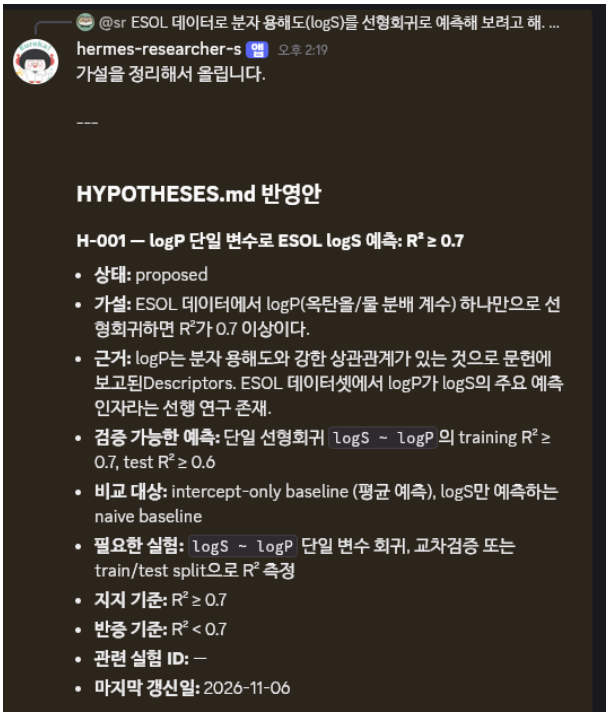
Discord에서 보낼 메시지

ESOL 데이터로 분자 용해도(logS)를 선형회귀로 예측해 보려고 해. 가설: logP 하나만으로도 R^2가 0.7 이상이고, 분자량·회전 결합 수·방향족 비율을 더하면 더 좋아진다. 가설로 정리해줘

PI가 `HYPOTHESES.md`에 넣을 반영안(`H-001`, `H-002`)을 가설·근거·검증 방법·지지/반증 기준으로 정리해 보여 줍니다. 아직 파일에 쓰지는 않았습니다.



내가 보낸 메시지

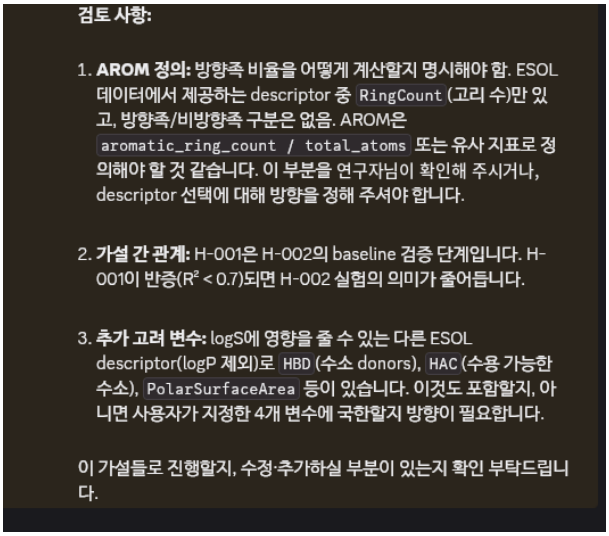


PI의 가설 반영안 (H-001)

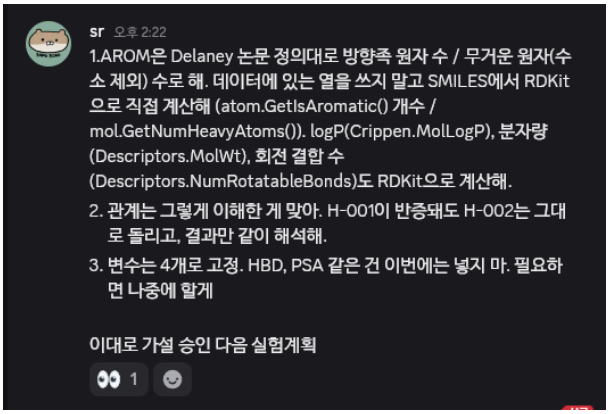
② 직원의 질문에 답하고 승인

반영안 끝에 **검토 사항**이 붙어 옵니다. 정의가 애매한 것(AROM을 어떻게 계산할지), 가설끼리의 관계, 변수를 더 넣을지처럼 **연구자가 정할 것**을 먼저 묻습니다. 무난하게 가려면 논문 정의를 그대로 쓰고 변수를 고정한다고 답한 뒤, 같은 메시지에서 승인합니다.

> **Discord에서 보낼 메시지**
>
> 1. AROM은 Delaney 논문 정의대로 방향족 원자 수 / 무거운 원자(수소 제외) 수로 해. 데이터에 있는 열을 쓰지 말고 SMILES에서 RDKit으로 직접 계산해 (atom.GetIsAromatic() 개수 / mol.GetNumHeavyAtoms()). logP(Crippen.MolLogP), 분자량(Descriptors.MolWt), 회전 결합 수(Descriptors.NumRotatableBonds)도 RDKit으로 계산해.
> 2. 관계는 그렇게 이해한 게 맞아. H-001이 반증돼도 H-002는 그대로 돌리고, 결과만 같이 해석해.
> 3. 변수는 4개로 고정. HBD, PSA 같은 건 이번에는 넣지 마.
>
> 이대로 가설 반영 승인. 다음은 실험 계획 세워줘.



PI의 검토 사항

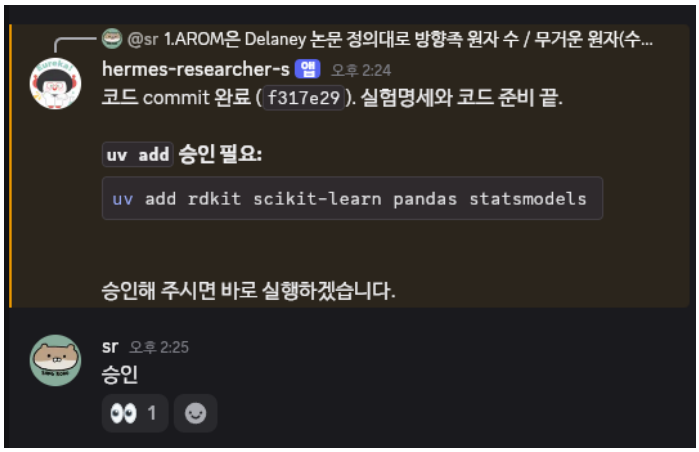


내 답과 승인

③ 패키지 설치 승인

PI가 실험 명세와 코드를 준비해 commit한 뒤, 새 Python 패키지가 필요하면 `uv add` 승인을 묻습니다(8.3절의 승인 규칙).

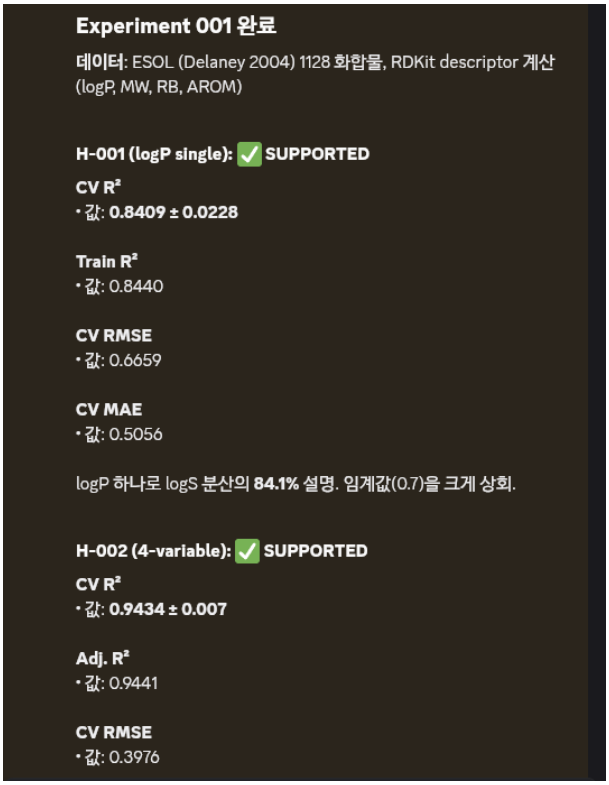
> **Discord에서 보낼 메시지**
> 승인



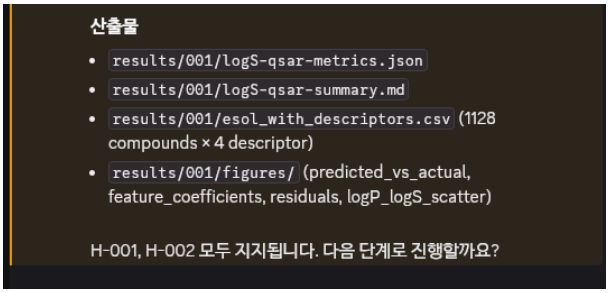
코드 commit 후 `uv add` 승인 요청

④ 결과 보고

승인하면 바로 실행하고, 끝나면 가설별로 지지/반증과 수치를 보고합니다. 이번에는 logP 하나로 R^2 약 0.84, 지표 4개로 약 0.94가 나와 두 가설 모두 지지되었습니다. 산출물은 `results/001/`에 모입니다(요약 `summary.md`, 수치 `metrics.json`, 그림 `figures/`).



가설별 결과 (CV R^2, RMSE 등)

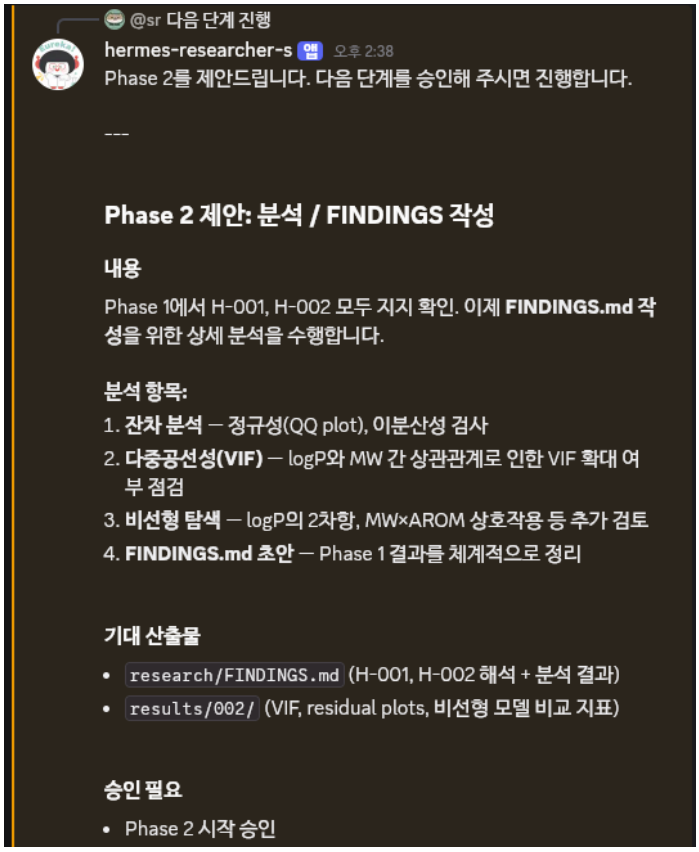


산출물 위치와 다음 단계 질문

⑤ 다음 단계

> **Discord에서 보낼 메시지**
> 다음 단계 진행

PI가 다음 단계(잔차 분석, 다중공선성, 비선형 항 탐색, `FINDINGS.md` 초안)를 제안하고 다시 승인을 묻습니다. 계속하려면 승인, 여기서 멈추려면 그냥 두면 됩니다.



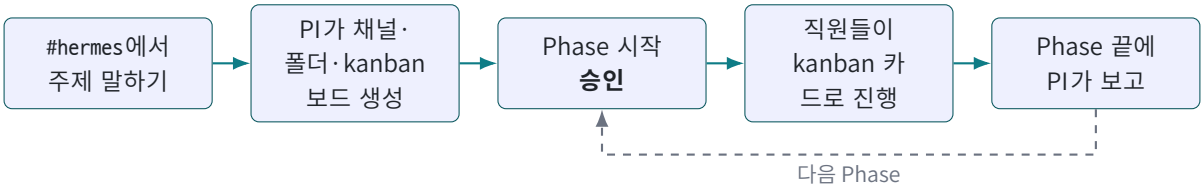
다음 단계 제안과 승인 요청

이 예시에서 볼 것 직원은 정의가 애매하면 **먼저 묻고**, 가설 반영·패키지 추가·다음 단계처럼 방향이 바뀌는 곳에서만 승인을 받습니다. 그 사이의 코드 작성·실행·정리는 알아서 합니다. 진행 중에 `상태 알려줘`를 보내면 언제든 현재 단계를 표로 받아 볼 수 있습니다.

CHAPTER 8

매일 쓰는 법

8.1 기본 흐름



내가 하는 일은 대부분 **주제를 말하고, Phase 시작을 승인하고, 보고를 읽는 것**입니다. Phase 시작을 승인하면 그 안의 kanban 카드·검토·commit·계획된 실험 실행은 PI가 대신 승인합니다.

8.2 상황별 방법

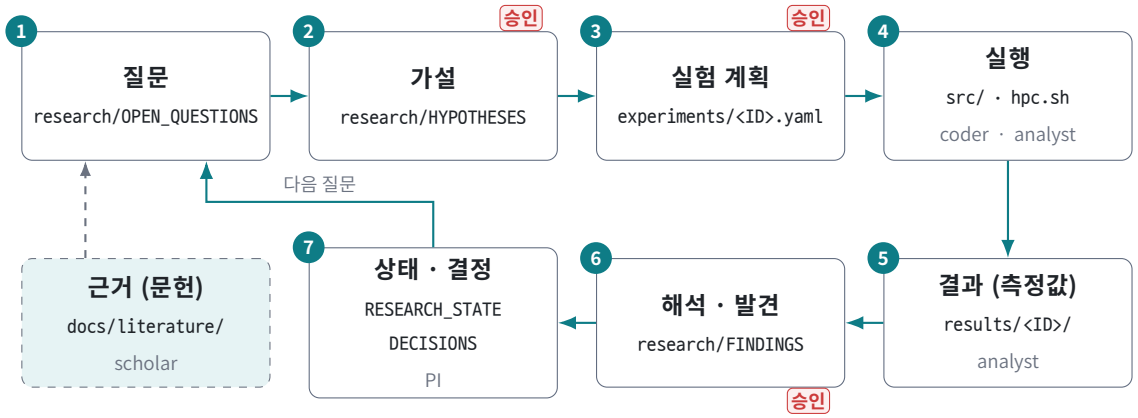
하고 싶은 것	방법
새 연구 주제 시작	#hermes에서 PI에게 주제를 말합니다 → 이름·태그 확인 → 30초쯤 뒤 새 채널에서 **내가 첫 메시지**를 보냅니다 (반응이 없으면 그 채널에서 /reset)
진행 상황·오늘 할 일	PI 봇의 /brief (프로젝트 채널은 그 프로젝트만, #hermes는 전체), 매일 09:00 자동 브리핑, 또는 진행 상황 알려줘
특정 직원에게 직접	@analyst ...처럼 멘션합니다. PI는 끼어들지 않습니다
내가 승인할 것	FINDINGS·HYPOTHESES·목표 변경, 다음 Phase 시작, 계획에 없는 실험, 자원 한도 초과, 삭제·되돌리기, 패키지 추가(uv add)
보고 받는 때	Phase가 끝날 때와 막혔을 때만. 중간이 궁금하면 묻거나 /brief
막힌 일	2번 막히면 차단 감시가 Claude 진단 결과를 #hermes에 올립니다. "연구자 결정 필요"에만 답합니다
규칙(SOUL) 고치기	souls/<역할>.md를 고칩니다 → bash install_souls.sh → 그 채널에서 그 봇에게 /reset. PI 핵심 규칙 souls/pi-rules.md는 고치면 바로 반영됩니다
설정 전체 점검	python3 tools/lab.py check
계산 노드에서 GPU 실험	bash tools/hpc.sh run <프로젝트 폴더> \"<명령>\" -p <파티션> --gres=gpu:1 (8.4절)
프로젝트 정리	python3 newproject.py remove <이름>. 채널은 archive-<이름>으로, 폴더는 _archive/로 보관합니다. 미리보기는 --dry-run
업데이트 받기	cd ~/Projects/hermes-team && git pull && bash extras/install_extras.sh && bash install_souls.sh

8.3 프로젝트 폴더 안내

이 단계에서 하는 일 PI가 새 프로젝트를 만들면 생기는 `~/Projects/<프로젝트>/` 안의 폴더가 각각 무엇이고, 누가 쓰고, 내가 볼 곳은 어디인지 정리합니다.

8.3.1 왜 이렇게 나눴나: 연구자의 생각 순서를 폴더로

이 폴더 구조는 연구자가 머릿속에서 하는 일 — **질문을 세우고, 가설을 세우고, 실험으로 확인하고, 결과를 해석해 다음 결정을 내리는 순환** — 을 그대로 파일로 옮긴 것입니다. 순환의 한 단계가 폴더(또는 파일) 하나입니다.



이렇게 나눈 이유는 네 가지입니다.

- **AI 직원은 대화가 끝나면 잊습니다.** 새 대화를 열거나 `/reset`하면 앞의 맥락이 사라집니다. 그래서 "지금 어디까지 왔고 무엇을 믿고 있나"를 파일로 남겨, 어느 직원이 언제 이어받아도 같은 출발점에서 시작하게 했습니다. `AGENTS.md`가 작업 전에 `RESEARCH_STATE`·`HYPOTHESES`부터 읽게 하는 것도 이 때문입니다.
- **측정값과 해석을 나눕니다.** `results/`에는 숫자만, `FINDINGS`에는 그 숫자로 무엇을 말할 수 있는지(관찰·해석·한계)를 따로 적습니다. AI가 그럴듯한 해석을 사실처럼 쓰는 일을 막고, 나중에 해석만 고쳐 쓸 수 있습니다.
- **승인은 연구 방향이 바뀌는 곳에만 둡니다.** 가설을 세우거나 바꿀 때, 실험을 실제로 돌릴 때, 발견을 기록할 때(그림의 **승인**)만 연구자에게 묻고, 나머지 반복 작업은 직원들이 알아서 합니다. 그래서 내 일은 "판단"에 집중됩니다.
- **같은 실험을 다시 돌릴 수 있게 합니다.** 실험 명세에 실행 명령·코드 버전(commit)·성공 기준을 적고, 결과를 같은 실험 ID 폴더에 모읍니다. 몇 달 뒤에도 "어떤 코드로, 어떤 설정으로 나온 숫자인지" 추적할 수 있습니다.

8.3.2 폴더별 안내

새 프로젝트 폴더에는 아래 항목이 생깁니다. 대부분은 직원들이 규칙(`AGENTS.md`)에 따라 채우므로, 내가 직접 고칠 곳은 거의 없습니다.

경로	무엇	누가 쓰나
규칙과 설정		
`AGENTS.md`	이 프로젝트의 규칙: 이름·태그, 먼저 읽을 파일, 자동 허용/승인 필요/금지, 코드 위임 방법	PI가 만들 때 작성

경로	무엇	누가 쓰나
`config/`	`project.yaml`(목적·실행 환경), `policy.yaml`(승인 규칙), `resource_limits.yaml`(GPU·시간 한도)	처음 그대로. 바꿀 때는 나
연구 기록 — 내가 가장 자주 읽는 곳		
`research/RESEARCH_STATE.md`	목표, 현재 단계, 다음 행동, 최근 실험, 결정 기록	PI (승인된 내용 반영)
`research/HYPOTHESES.md`	가설과 예측·반증 조건 (`H-001` …)	수정안은 직원, **반영은 내 승인 후**
`research/FINDINGS.md`	확인된 결과: 관찰·해석·한계를 나눠 기록 (`F-001` …)	analyst 수정안 → **내 승인 후**
`research/OPEN_QUESTIONS.md`	아직 풀리지 않은 질문과 우선순위	직원 제안 → 내 승인
`research/DECISIONS.md`	중요한 설계 결정과 이유	PI
실험		
`experiments/`	실험 명세 `<실험ID>.yaml`: 무엇을, 어떤 명령으로, 성공 기준, 승인 상태. `templates/`는 빈 양식	analyst 작성, **실행은 내 승인 후**
`schemas/`	실험 명세가 지켜야 할 형식(JSON Schema)	처음 그대로
`results/<실험ID>/`	실험 결과: `summary.md`, `metrics.json`, 그림(`figures/`)	analyst
`plots/`	결과 그림을 그리는 코드 (`plot_<실험ID>.py`). `results/`만 읽음	analyst가 Claude Code에 맡김
코드와 Python 환경		
`src/`	실험 코드 (필요해지면 생깁니다)	coder가 Claude Code에 맡김
`scripts/researchctl`	점검 도구: `doctor`(환경 점검·수리), `list`·`validate`(명세 확인)	직원이 실행
`pyproject.toml`, `uv.lock`	이 프로젝트의 Python 패키지 목록과 정확한 버전	`uv add` (패키지 추가는 **내 승인**)
`.venv/` (숨김)	실제 설치된 환경. git에 올리지 않음	`uv`가 자동
문서		
`docs/literature/`	문헌 조사 보고서, `references.bib`, 내가 구한 PDF(`pdfs/`)	scholar. **PDF는 내가 넣어도 됨**
`docs/paper/`	논문 초안 (필요해지면 생깁니다)	writer가 Claude Code에 맡김
`docs/*_ko.md`	이 템플릿의 설계 설명서 (구조, 운영, 보안, 용어)	읽기용
예시와 참고 (처음에는 건드리지 않음)		
`deployment/`	Slurm·systemd·cron 예시 파일	참고용. GPU 실행은 8.4절 `hpc.sh`
`monitoring/`	실험 기록·관측 안내	참고용
`skills/`	반복 작업 절차(스킬) 양식, 예: 결과 분석 절차	직원이 참고
`README_ko.md`	이 템플릿 자체의 소개 (예시 프로젝트 이름이 섞여 있음)	참고용

매일 볼 곳은 세 군데 ① `research/RESEARCH_STATE.md` — 지금 어디까지 왔고 다음에 뭘 하는지. ② `research/FINDINGS.md` — 지금까지 확인된 결과. ③ `results/<실험ID>/summary.md` — 방금 끝난 실험 하나의 요약. 나머지는 PI의 보고나 `/brief`로 충분합니다.

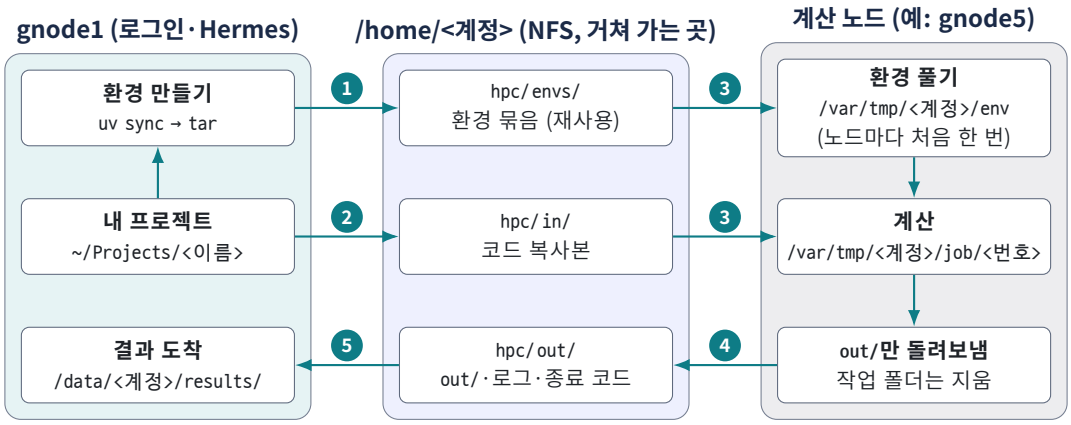
승인이 필요한 곳은 정해져 있습니다 `research/`의 가설·결과 수정, 실험 실행(명세의 `approval`), 패키지 추가(`uv add`), 파일 삭제는 직원이 먼저 Discord에서 묻습니다. 자세한 목록은 그 프로젝트의 `AGENTS.md` "권한 규칙"과 `config/policy.yaml`에 있습니다.

8.4 계산 노드에서 실험 돌리기 (hpc.sh)

이 단계에서 하는 일 GPU 실험을 Slurm으로 계산 노드(예: gnode5)에 보내고, 결과를 gnode1의 `/data`로 받아 옵니다.

gnode1의 `/ai`와 `/data`는 다른 노드에서 보이지 않습니다. 그래서 `~/Projects`에서 `sbatch`를 바로 하면 계산 노드가 로그 파일을 열지 못해 job이 시작도 못 하고 끝납니다(`RaisedSignal:53`). 또 계산 노드는 인터넷이 안 되어 거기서 패키지를 설치할 수 없습니다. `tools/hpc.sh`가 이 일을 대신합니다.

	`hpc.sh run`이 하는 일
1	gnode1에서 uv로 Python과 패키지를 설치해 묶음 하나로 `/home/<계정>/hpc/envs/`에 둡니다. 패키지가 바뀔 때만 다시 만듭니다
2	프로젝트 폴더를 `/home/<계정>/hpc/in/`에 복사하고 `sbatch`로 제출합니다
3	계산 노드가 코드와 환경을 자기 로컬 디스크(`/var/tmp/<계정>`)에 풀어 계산합니다. 환경은 노드마다 처음 한 번만 풉니다
4	끝나면(실패해도) `out/` 폴더와 종료 코드만 `/home`으로 돌려놓고 로컬 작업 폴더를 지웁니다
5	gnode1이 결과를 `/data/<계정>/results/`로 옮기고 `/home`에 올렸던 것을 지웁니다



끝나면 `in/`·`out/`은 비워집니다. 환경 묶음만 남아 다음에 다시 씁니다.

8.4.1 처음 한 번

결과가 거쳐 가는 곳(`/home`)과 도착하는 곳(`/data`)을 만들고, 쓸 **파티션 이름**을 확인합니다.

```
mkdir -p /home/$USER/hpc /data/$USER/results && echo OK
sinfo -s
```

✓ **성공하면** OK가 나오고, sinfo -s의 PARTITION 칸에 파티션 이름(예: p6k1)이 보입니다.

파티션과 노드는 다릅니다 gnode5 같은 것은 **노드**(컴퓨터) 이름이고, -p에는 노드 묶음인 **파티션** 이름을 씁니다. -p gnode5라고 쓰면 invalid partition으로 거절됩니다. 어떤 노드가 어느 파티션에 있고 GPU가 몇 장인지는 sinfo -N -n gnode5 -O NodeList,Partition,Gres로 봅니다. 특정 노드에서 돌리고 싶으면 -w gnode5를 더합니다.

mkdir: Permission denied가 나오면 /data/<계정> 폴더가 없고 만들 권한도 없는 것입니다. 관리자에게 gnode1의 /data/<계정>을 만들어 달라고 부탁합니다. 다른 곳에 결과를 받으려면 echo 'export HPC_RESULTS=<쓸 수 있는 폴더>/results' >> ~/.bashrc 후 source ~/.bashrc.

8.4.2 시험해 보기

패키지 없는 작은 프로젝트로 한 번 돌려 봅니다. 계산 노드 이름과 GPU가 보이면 준비가 끝난 것입니다.

```
mkdir -p ~/Projects/hpctest && cd ~/Projects/hpctest && uv init -q --bare
cat > hello.py <<'END'
import os, socket, platform
os.makedirs("out", exist_ok=True)
with open("out/hello.txt", "w") as f:
    f.write(f"host={socket.gethostname()}\npython={platform.python_version()}\ncwd={os.getcwd()}\n")
END
bash ~/Projects/hermes-team/tools/hpc.sh run ~/Projects/hpctest \
  'python hello.py; nvidia-smi > out/gpu.txt; echo "CUDA_VISIBLE_DEVICES=$CUDA_VISIBLE_DEVICES" >>
     out/hello.txt' \
  -p <파티션> --gres=gpu:1 -t 00:05:00
cat /data/$USER/results/hpctest-*/hello.txt
```

```
host=gnode5
python=3.15.2
cwd=/var/tmp/minji/job/999
CUDA_VISIBLE_DEVICES=0
```

✓ **성공하면** host가 gnode1이 아닌 계산 노드이고, cwd가 계산 노드의 로컬 디스크(/var/tmp)이며, CUDA_VISIBLE_DEVICES에 숫자 하나가 있습니다. gpu.txt에 GPU 표가 있습니다.

nvidia-smi에 GPU가 여러 장 보여도 괜찮습니다 노드의 GPU가 모두 보일 수 있지만, 내 job이 쓸 GPU는 CUDA_VISIBLE_DEVICES가 정합니다. PyTorch 등은 이 값을 따르므로 --gres=gpu:1이면 1장만 씁니다.

8.4.3 실행

실행 명령은 따옴표로 묶고, 뒤에는 평소 sbatch에 주던 옵션을 그대로 씁니다.

```
bash ~/Projects/hermes-team/tools/hpc.sh run ~/Projects/<프로젝트> "python train.py" \
  -p <파티션> --gres=gpu:1 -t 01:00:00
```

```
[hpc] 환경 만들기: /var/tmp/minji/env/<프로젝트>-3f9a1c2b7d4e
[hpc] 코드 올리기: /home/minji/hpc/in/<프로젝트>-1105-142233-5120
[hpc] 제출: job 931 (<프로젝트>-1105-142233-5120)
job 931 → /data/minji/results/<프로젝트>-1105-142233-5120-931  (exit 0)
```

✓ **성공하면** 마지막 줄이 `(exit 0)`입니다. 결과 폴더에 `out/`에 쓴 파일, 화면 출력 `run.log`, Slurm 로그 `slurm.log`, `exit_code`가 있습니다.

명령	하는 일
`hpc.sh run <폴더> "<명령>" [옵션]`	제출하고 끝날 때까지 기다렸다가 결과를 가져옵니다
`hpc.sh submit <폴더> "<명령>" [옵션]`	제출만 합니다. 긴 실험은 이것으로
`hpc.sh status`	돌고 있는 job과 아직 안 가져온 결과
`hpc.sh fetch <job번호|all>`	끝난 job의 결과를 가져옵니다

규칙 두 가지 ① 프로그램은 결과를 **`out/` 폴더에** 써야 돌아옵니다. 다른 곳에 쓴 파일은 job이 끝나면 지워집니다. `results/`처럼 다른 폴더에 쓰는 코드라면 명령 끝에 복사를 붙입니다. 예: `"python src/logS_qsar.py && cp -r results out/"` (7.4절 실험을 계산 노드에서 다시 돌린 명령).
② 데이터셋은 계산 노드에서 보이지 않습니다. 필요한 데이터는 프로젝트 폴더 안에 두거나, 관리자와 정한 방법으로 옮깁니다.

실패했을 때 `exit` 숫자가 0이 아니면 결과 폴더의 `run.log`(프로그램 오류)와 `slurm.log`(노드·환경 문제)를 봅니다. `exit 90`은 계산 노드 로컬 디스크가 20GB보다 적게 남아 시작하지 않은 것입니다.

걸리는 시간 패키지를 바꾼 뒤 첫 실행만 환경을 다시 묶느라 몇 분 걸립니다(torch를 넣은 환경에서 약 5분). 패키지가 그대로면 환경을 다시 만들지도 풀지도 않아서 제출부터 결과까지 몇 초입니다(같은 환경 두 번째 실행 3초). 그래서 패키지를 이것저것 바꿔 보는 동안은 gnode1에서 `uv add`와 `uv run`으로 시험하고, 패키지가 정해진 뒤 계산 노드로 보냅니다.

8.5 새 직원 추가하기

이 단계에서 하는 일 6명 말고 직원을 더 둡니다. 예: 통계 검토만 맡는 `stats`.

설치 때 한 일을 **그 직원 한 명에게만** 다시 하는 것입니다. 아래에서 `stats`를 새 이름으로 바꿔 읽습니다. 이름은 영어 소문자로 짓습니다.

1. **Discord 봇 만들기.** 4.1절을 그대로 한 번 더 합니다. Application ID, 토큰, 봇 ID를 기록표 빈 줄에 적습니다.
2. **profile 만들고 토큰 넣기.** 4.3절처럼 PI를 복제한 뒤 자기 봇 토큰을 넣습니다.

```
hermes profile create stats --clone --description "통계 검토 담당. \
분석 결과의 검정 방법과 수치를 확인해 보고한다. 직접 수정하지 않는다."
hermes -p stats gateway setup        # Discord → stats 봇 토큰 → y → Enter
hermes -p stats chat -q "OK라고만 답해"
```

3. **모델 확인.** PI를 복제했으므로 모델 설정도 PI와 같습니다. 주 모델은 로컬 모델이고, 예비 모델은 Claude입니다. 두 줄로 확인합니다.

```
hermes -p stats config get model.default     # qwen3.6-35b-a3b
hermes -p stats fallback list                # 1. claude-opus-5-5
```

ChatGPT를 주 모델로 쓰려면 4.3.3절의 로그인 한 줄과 모델 두 줄을 `-p stats`로 실행합니다.

4. **역할(SOUL) 쓰기.** 비슷한 직원의 파일을 복사해 고칩니다. PI가 일을 맡기려면 PI도 이 직원을 알아야 합니다.

```
cd ~/Projects/hermes-team
cp souls/reviewer.md souls/stats.md
nano souls/stats.md          # 이름·담당·할 일·하지 않을 일을 고칩니다
nano souls/pi.md             # 맨 위 직원 표에 stats 줄을 하나 추가합니다
nano souls/_team_ids.md      # "- scholar: ..." 아래에 - stats: `<@봇ID>` 줄 추가
```

5. **팀 설정에 넣기.** `team.json`의 `"profiles"` 목록 끝에 `"stats"`를 더합니다. 앞 항목 뒤에 쉼표를 잊지 않습니다. 생각 깊이를 정하려면 `"reasoning"`에도 `"stats": "medium"`을 더합니다.

team.json (바뀐 부분)

```
  "profiles": ["default", "coder", "analyst", "reviewer", "writer", "scholar", "stats"],
```

6. **적용.** 6.2절 네 줄을 그대로 다시 실행합니다. 이번엔 새 직원까지 포함됩니다.

```
python3 tools/lab.py env
python3 tools/lab.py reasoning
python3 newproject.py sync
bash install_souls.sh
```

7. **슬래시 명령 등록.** 6.3절을 stats 한 명에게만 합니다.

```
echo 'DISCORD_COMMAND_SYNC_POLICY=bulk' >> ~/.hermes/profiles/stats/.env
hermes gateway restart --all
sleep 30
sed -i '/^DISCORD_COMMAND_SYNC_POLICY=bulk$/d' ~/.hermes/profiles/stats/.env
hermes gateway restart --all
```

✓ **성공하면** `python3 tools/lab.py check`에 stats가 나오고 문제가 없습니다. 프로젝트 채널에서 `@stats 넌 누구야?`에 새 역할로 답합니다.

원래 있던 직원은 건드리지 않습니다 위 단계는 모두 새 직원 한 명에게만 하거나, 6명에게 다시 해도 결과가 같은 명령입니다. 기존 직원의 설정과 기억은 그대로입니다.

CHAPTER 9

문제 해결

9.1 먼저 해 볼 것

```
python3 ~/Projects/hermes-team/tools/lab.py check     # 무엇이 틀렸는지 + 고치는 명령
hermes gateway status                                 # 게이트웨이가 도는지
hermes doctor                                         # Hermes 자체 진단
```

로그 위치: PI는 `~/.hermes/logs/`, 직원은 `~/.hermes/profiles/<이름>/logs/`.

9.2 증상별 해결

증상	원인 / 해결
설치	
`hermes: command not found`	`source ~/.bashrc` 또는 터미널을 새로 엽니다
`install_souls.sh`가 "채우지 않은 ID"	6.1절 `team.json`, `_team_ids.md`의 빈 자리를 채웁니다
`git switch main` 실패	저장소 기본 브랜치가 `master`입니다. 2.1절 `init.defaultBranch main`을 해 두면 새 저장소는 `main`이 됩니다
Discord 봇	
자동완성에 봇이 안 보임	서버에 초대되지 않았습니다 → 4.1절 4번을 **그 봇의** Application ID로 다시
멘션했는데 반응이 없음	역할(role)을 골랐습니다 → **APP** 표시 항목을 고르거나 `<@봇ID>`를 직접 입력
봇이 오프라인 / 하나를 켜면 다른 게 꺼짐	토큰 중복입니다 (`--clone`이 PI 토큰을 복사) → `lab.py check`로 확인 → 4.3.2절 다시
봇이 접속하지 못함	Message Content Intent가 꺼져 있습니다 → 4.1절 3번
PI가 멘션해도 직원이 무시함	협업 설정 → `python3 tools/lab.py env` 후 `hermes gateway restart --all`
다른 직원만 멘션했는데 PI도 길게 답함	정상은 – 한 글자입니다. PI SOUL이 옛것 → `install_souls.sh` + `/reset`
새 프로젝트 만들 때 HTTP 403	6.4절 카테고리 권한, `team.json`의 `category_id` 확인
모델	
답은 오는데 `/slots`의 `id_task`가 그대로	로컬 모델이 꺼져 Claude 예비가 답하는 중입니다 → 4.2.1절. 오래 두면 Claude 사용량이 빨리 찹니다
429 / rate limit	ChatGPT 사용 한도입니다. kanban 카드는 대기했다가 자동으로 다시 시작됩니다. 기다리면 됩니다
동작	

증상	원인 / 해결
규칙을 바꿨는데 예전처럼 행동함	열린 대화는 예전 SOUL을 가지고 있습니다 → 그 채널에서 그 봇에게 `/reset`
`config.yaml`을 고쳤는데 그대로	`# discord:`로 시작하는 주석 예시를 고친 것입니다. discord 설정은 `newproject.py sync`가 관리합니다
kanban 카드 결과 파일이 사라짐	작업 위치가 `scratch`였습니다 → 항상 `dir:<프로젝트 경로>` (PI SOUL에 규칙 있음)
kanban 카드가 `blocked`	같은 원인으로 2번 실패했습니다. 원인을 고친 뒤 `hermes kanban --board <p> unblock <id>`
브리핑 로그에 `BLOCKED: script execution via -e/-c`	무인 실행에서 `python -c` 같은 명령이 막힌 것입니다. 다른 방법으로 다시 시도하므로 무시해도 됩니다
`uv` 없음 / `.venv` 깨짐	그 프로젝트 폴더에서 `bash scripts/researchctl doctor --fix`

CHAPTER A

지금 연구실을 다른 컴퓨터로 옮기기

같은 봇과 같은 프로젝트를 그대로 쓰는 경우입니다. 새로 만들지 않으므로 3~6장 대부분이 필요 없습니다.

1. **새 컴퓨터**에서 2.1~2.5절을 합니다 (도구 설치, Claude · Antigravity 로그인). 로컬 모델 서버도 새 컴퓨터에서 돌고 있어야 합니다.
2. **원래 컴퓨터**에서 백업합니다. 끝날 때 나오는 **빠진 파일 보고**를 확인합니다.

```
hermes backup                          # ~/hermes-backup-<시각>.zip (설정, 토큰, 기억, profile 전부)
tar czf ~/projects.tgz -C ~ Projects   # hermes-team + 모든 연구 프로젝트
```

3. **원래 컴퓨터의 게이트웨이를 멈춥니다.** 같은 토큰으로 두 곳이 동시에 접속하면 안 됩니다.
4. 두 파일을 새 컴퓨터로 복사한 뒤 복원합니다.

```
hermes import ~/hermes-backup-<시각>.zip
tar xzf ~/projects.tgz -C ~
```

5. **사용자 이름(홈 경로)이 바뀌었다면** 프로젝트 경로를 고치고 채널 연결을 다시 만듭니다.

```
cd ~/Projects/hermes-team
sed -i "s#/home/<예전사용자>/#$HOME/#g" projects.json
python3 newproject.py sync
```

6. 로컬 모델 포트가 바뀌었다면 5.2절 “로컬 모델 포트가 바뀌었을 때” 명령으로 4명의 주소를 고칩니다.
7. 권한과 자동 장치를 설치합니다.

```
python3 tools/lab.py agy-perms
bash extras/install_extras.sh
bash install_souls.sh
python3 tools/lab.py check
```

8. 6.8절(재부팅 대비)을 하고 7.2절 시험을 합니다.

> **백업 파일은 옮긴 뒤 지웁니다** `hermes backup` zip에는 **봇 토큰과 로그인 정보**가 들어 있습니다.

CHAPTER B

다른 사람에게 넘기기 (넘겨주는 쪽)

hermes-team 폴더 자체가 GitHub 저장소입니다. 저장소는 두 개로 나눠 씁니다.

- **개발용** hermes-team (비공개): 평소에 고치고 push하는 곳. commit 기록이 모두 남습니다.
- **배포용** hermes-team-public (공개): 남에게 주는 곳. commit 기록 없이 **최신 1개**만 있습니다.

배포용을 최신으로 바꿀 때는 개발용에서 commit한 뒤 아래 한 줄을 실행합니다. main에 commit된 파일만 보내고, 보내기 전에 개인 정보를 검사해 섞여 있으면 멈춥니다.

```
bash ~/Projects/hermes-team/tools/publish.sh
```

받는 사람에게는 배포용 주소와 이 문서를 주면 2장부터 따라 합니다 (git clone https://github.com/<계정>/hermes-team-public.git ~/Projects/hermes-team).

- **저장소에 올라가는 것**: SOUL · 템플릿 원본(연구자 이름은 {{연구자}}), 스크립트, 플러그인, 문서
- **올라가지 않는 것**: team.json, souls/_team_ids.md, projects.json, memories/, state/, logs/, 슬라이드
- commit할 때 tools/git-hooks/pre-commit이 연구자 이름 · Discord ID · 홈 경로 · 위 파일을 검사해서 섞이면 막습니다.
- 인터넷 없이 파일로 넘겨야 할 때만 bash tools/make_handover.sh로 ~/Projects/hermes-team-share.tar를 만듭니다.

> **내 연구실을 다시 설치할 때** 직원들이 쌓은 장기 기억은 다시 설치하기 전에 bash tools/memories.sh export로 받아 두고, 설치 후 bash tools/memories.sh import → bash install_souls.sh로 되돌립니다. 새 연구실이면 필요 없습니다.

CHAPTER C

명령어와 파일 요약

C.1 자주 쓰는 명령

명령	뜻
`python3 tools/lab.py check`	전체 점검 (아무것도 바꾸지 않음)
`hermes gateway status`	게이트웨이 상태
`hermes gateway restart --all`	6명 모두 다시 켜기
`hermes -p <직원> chat -q "..."`	터미널에서 직원 한 명에게 한 번 묻기
`hermes profile list`	직원(profile) 목록
`hermes kanban --board <프로젝트> list`	프로젝트의 kanban 카드 목록
`hermes cron status`	아침 브리핑 등 예약 작업
`python3 newproject.py list`	등록된 프로젝트·채널
`python3 newproject.py sync`	채널 연결 다시 만들기
`bash install_souls.sh`	규칙(SOUL) 설치 + 재시작
`bash extras/install_extras.sh`	자동 장치 설치
`bash set_name.sh <이름>`	연구자 이름 바꾸기

C.2 파일 지도

어디에 무엇이 있나

```
~/Projects/hermes-team/          ← 연구실 "설계도" (이 폴더만 옮기면 같은 연구실을 만들 수 있다)
├── INSTALL.md / INSTALL.tex     이 문서
├── LOCAL_MODEL.md               로컬 모델 설치
├── team.json                    Discord ID, 직원 목록, 생각 깊이 (내 PC 전용)
├── projects.json                프로젝트 ↔ 채널 (newproject.py가 관리, 내 PC 전용)
├── newproject.py                새 프로젝트: 폴더+git+uv, 채널, 채널 연결, kanban 보드
├── install_souls.sh             souls/*.md → 6명의 SOUL.md, 재시작
├── set_name.sh                  연구자 이름 정하기
├── souls/                       직원별 역할·규칙 원본 + _team_ids.md (ID)
├── template/                    새 프로젝트 뼈대
├── tools/lab.py                 설치 도우미: env / reasoning / agy-perms / rename / check
├── tools/hpc.sh                 계산 노드 실행: run / submit / status / fetch (hpc_job.sh와 한 쌍)
├── extras/                      자동 장치 원본 (install_extras.sh가 설치)
└── memories/ state/ logs/       직원 기억 사본, 실행 상태, 기록 (내 PC 전용)

~/.hermes/                       ← Hermes 데이터 (비밀 포함, git에 올리지 않는다)
├── config.yaml  .env  SOUL.md   PI(default)
└── profiles/<직원>/             직원별 config.yaml, .env(봇 토큰), SOUL.md, logs/

~/Projects/<프로젝트>/           ← 연구 프로젝트 하나 (git 저장소)
├── AGENTS.md                    이 프로젝트에서 직원들이 따를 규칙
├── research/                    RESEARCH_STATE, HYPOTHESES, FINDINGS (승인 후에만 수정)
└── experiments/ results/ src/ docs/ scripts/researchctl

/data/<계정>/results/    ← hpc.sh로 계산 노드에서 돌린 실험 결과
/home/<계정>/hpc/                ← hpc.sh가 계산 노드와 주고받는 임시 칸 (공유 디스크, 비워 둔다)
```